

SECURE WEB LINK ANALYZER: A HYBRID RULE-BASED AND MACHINE-LEARNING APPROACH FOR URL RISK ASSESSMENT

FINAL YEAR

PROJECT PHASE-I REPORT

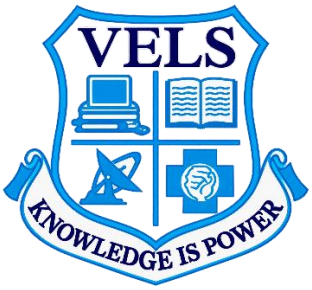
Submitted by

SEENIVASAN E-23619134

HARISH SAGAR S-23619115

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS) CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"SECURE WEB LINK ANALYZER: A HYBRID RULE-BASED AND MACHINE-LEARNING APPROACH FOR URL RISK ASSESSMENT"** is the bonafide work of **SEENIVASAN E - 23619134** and **HARISH SAGAR S - 23619115**, who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on these or any other candidates.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya, M.E., Ph.D.

Associate Professor & Head

Department of AI & ML

School of Engineering

VISTAS

Chennai-600 117

SIGNATURE OF THE SUPERVISOR

Mrs. V.P. Anitha,M.Tech,Ph.D.

Associate Professor

Department of AI & ML

School of Engineering

VISTAS

Chennai-600 117

Submitted for the Project Work and Viva-Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai - 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Dr. Ishari K. Ganesh**, and it is with great pleasure that we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Ms. S.J. Kavitha, Assistant Professor, Department of AI & ML**, who has patiently guided us and provided invaluable advice and help throughout the work. We are thankful to our **Project Coordinator, Ms. V.P.Anitha**

We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

SEENIVASAN E-23619134

HARISH SAGAR S-23619115

ABSTRACT

Secure Web Link Analyzer, presented as URLNivox, assesses warning patterns in a URL before its destination is opened. The project combines explicit rules, structural heuristics, and a locally stored Random Forest classifier. It helps a user inspect a link and understand the reasons for a warning. Runtime analysis does not retrieve the destination page, perform DNS lookups, follow redirects, or inspect page content.

The application uses a Flask backend, an HTML, CSS, and JavaScript interface, and SQLite for optional history. A shared pipeline extracts 24 numerical features. Direct rules examine credentials, IP destinations, unusual ports, encoded characters, executable-style paths, selected terms, and brand lookalikes. Structural checks measure length, query complexity, subdomains, hostname entropy, digits, and special characters. When the model is available, the final score combines 60% heuristic evidence and 40% model output; otherwise it reports an explicit heuristic fallback.

The classifier was trained on a balanced sample of 40,000 labelled URLs from the published DEPHIDES dataset. A split by estimated registered domain produced 32,118 training records and 7,882 test records, with no overlapping estimated domain groups. On this historical held-out subset, the classifier achieved 89.96% accuracy, 88.29% precision, 91.77% recall, an F1 score of 0.9000, and ROC AUC of 0.9677. These measurements describe the binary classifier, not the complete three-band hybrid decision system.

All 21 supplied automated tests passed during verification on 14 October 2026. The report documents the architecture, features, score calculation, dataset, model, API, output, and selected source code. The main outcome is a reproducible local prototype with traceable evidence. Historical labels, one data source, approximate domain grouping, and uncalibrated probabilities limit its conclusions. A low score indicates few observed URL warning patterns; it does not certify a destination as safe.

Keywords - URL analysis; phishing; Random Forest; lexical features; Flask; SQLite; pre-visit assessment.

TABLE OF CONTENTS

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **vii**

LIST OF TABLES ..... **viii**

LIST OF ABBREVIATIONS ..... **ix**

1 INTRODUCTION **1**

1.1 Overview of the Project 1
1.2 Motivation 2
1.3 Problem Statement 3
1.4 Objectives 4
1.5 Scope of the Project 5
1.6 Contributions 7
1.7 Organisation of the Report 8

2 LITERATURE REVIEW **9**

2.1 Literature Overview 9
2.2 Learned URL Representations 10
2.3 Published Dataset Foundation 11
2.4 Families of URL Assessment Methods 12
2.5 Rules and Machine Learning 13
2.6 Software Foundation 14
2.7 Limitations of Existing Work and Project Gap 14

3 SYSTEM DESIGN **17**

3.1 Existing System 17
3.2 Drawbacks of the Existing System 18
3.3 Proposed System 18
3.4 System Architecture 20
3.5 Architectural Flow 21
3.6 Module Description 23
3.7 Design Principles and Requirements 27
3.8 Software Requirements 30
3.9 Hardware Requirements 32
3.10 Project Timeline 32

4 IMPLEMENTATION **34**

4.1 Methodology 34
4.2 URL Validation and Redaction 35
4.3 Length and Destination Features 36
4.4 Obfuscation and Structural Features 38
4.5 Direct Rule Analysis 40
4.6 Structural Heuristics 41
4.7 Dataset Preparation 42
4.8 Random Forest Training 43
4.9 Model Loading and Failure Handling 45
4.10 Browser Interface and History 47
4.11 Source Code 49
4.12 Input 52
4.13 Output 53
4.14 Calculation 56

5 RESULTS AND DISCUSSION 58
5.1 Verification and Testing 58
5.2 Evaluation Protocol 59
5.3 Confusion Matrix and Error Analysis 62
5.4 Feature Importance 64
5.5 Reproducibility of the Results 65
5.6 Demonstration and Troubleshooting 66
5.7 Privacy and Robustness 68
5.8 Discussion of the Measured Outcome 69

6 CONCLUSION AND FUTURE WORK 70
6.1 Summary 70
6.2 Objectives Revisited 71
6.3 Limitations 71
6.4 Future Work 73
6.5 Conclusion 74

REFERENCES 75

APPENDIX A - REPRODUCIBILITY AND REPOSITORY LAYOUT 77

LIST OF FIGURES

Figure 1.1 URLNivox dashboard from the supplied project preview, showing single-link analysis and the active model. _____ 1

Figure 3.1 Application architecture and local dependencies _____ 20

Figure 3.2 Analysis flow with optional local persistence _____ 22

Figure 4.1 Class counts in the supplied training and test partitions _____ 44

Figure 4.2 Saved assessment history in the supplied project preview. Search, verdict filters, and report reopening support review._____48

Figure 4.3 Analysis output for the executable-style example. The preview displays a final score of 86, direct rule score 77, structure score 10, and ML score 94._____ 54

Figure 4.4 Filtered history from the supplied project preview. A search with no matching saved assessment shows an empty result state._____ 55

Figure 4.5 Components of the hybrid score with rounding applied in code_____ 56

Figure 5.1 Binary classifier confusion matrix on 7882 held out URLs _____ 62

Figure 5.2 Ten largest impurity based feature importances from the trained forest _____ 64

LIST OF TABLES

Table 1.1 Implemented claims and their evidence boundaries. 6
Table 2.1 Comparison of URL assessment approaches. 12
Table 2.2 Project design decisions and their consequences. 15
Table 3.1 Principal Python modules and responsibilities. 23
Table 3.2 Principal application use cases. 24
Table 3.3 History database fields and their meanings. 25
Table 3.4 Local API endpoints and response contracts. 26
Table 3.5 Functional requirements and observable acceptance evidence. 28
Table 3.6 Quality requirements and implementation boundaries. 29
Table 3.7 Recorded implementation and verification environment. 31
Table 3.8 Documented project and report checkpoints. 33
Table 4.1 Numerical URL features used by the classifier. 37
Table 4.2 Numerical URL features used by the classifier. 38
Table 4.3 Direct URL rules and assigned points. 40
Table 4.4 Structural heuristic conditions and points. 41
Table 4.5 Recorded dataset preparation counts. 42
Table 4.6 Class distribution in the training and test partitions. 44
Table 4.7 Model availability and failure handling. 46
Table 4.8 Runtime input contract for individual and batch analysis. 52
Table 4.9 Verified example outputs from the supplied application. 53
Table 4.10 Overall risk score bands in the application. 56
Table 5.1 Executed automated tests and representative coverage. 58
Table 5.2 Binary classifier performance on 7,882 held-out records. 60
Table 5.3 Troubleshooting checks for a local demonstration. 67
Table 6.1 Project objectives and observed outcomes. 71
Table 6.2 Limitations affecting interpretation and deployment. 72
Table 6.3 Proposed experiments and the questions they address. 73
Table A.1 Runtime configuration and artifact locations. 77
Table A.2 Files supplied with the runnable code package. 78

LIST OF ABBREVIATIONS

API	Application Programming Interface
AUC	Area Under the Curve
CSV	Comma Separated Values
CSP	Content Security Policy
CSS	Cascading Style Sheets
DNS	Domain Name System
F1	Harmonic mean of precision and recall
FN	False Negative
FNR	False Negative Rate
FP	False Positive
FPR	False Positive Rate
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP over a secure connection
IDNA	Internationalized Domain Names in Applications
IP	Internet Protocol
JSON	JavaScript Object Notation
KiB	Kibibyte
ML	Machine Learning
RF	Random Forest
ROC	Receiver Operating Characteristic
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SSRF	Server Side Request Forgery
TLD	Top Level Domain
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
VISTAS	Vels Institute of Science, Technology and Advanced Studies
WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Secure Web Link Analyzer is a final year software project that combines web development, data processing, supervised learning, and software testing. The browser interface is named URLNivox. It supports individual checks, small batches, explanatory reports, optional history, and a page showing the status and measured performance of the included model. The implementation is self contained after its Python dependencies are installed.

A purely numerical output would leave an important question unanswered: why did the system assign that score? To address this, the application preserves individual rule checks, structural signals, and model information separately. The report interface can show the direct evidence alongside the aggregate assessment. The ML component contributes a learned pattern estimate, while the rule component supplies reasons that can be traced to specific URL properties.

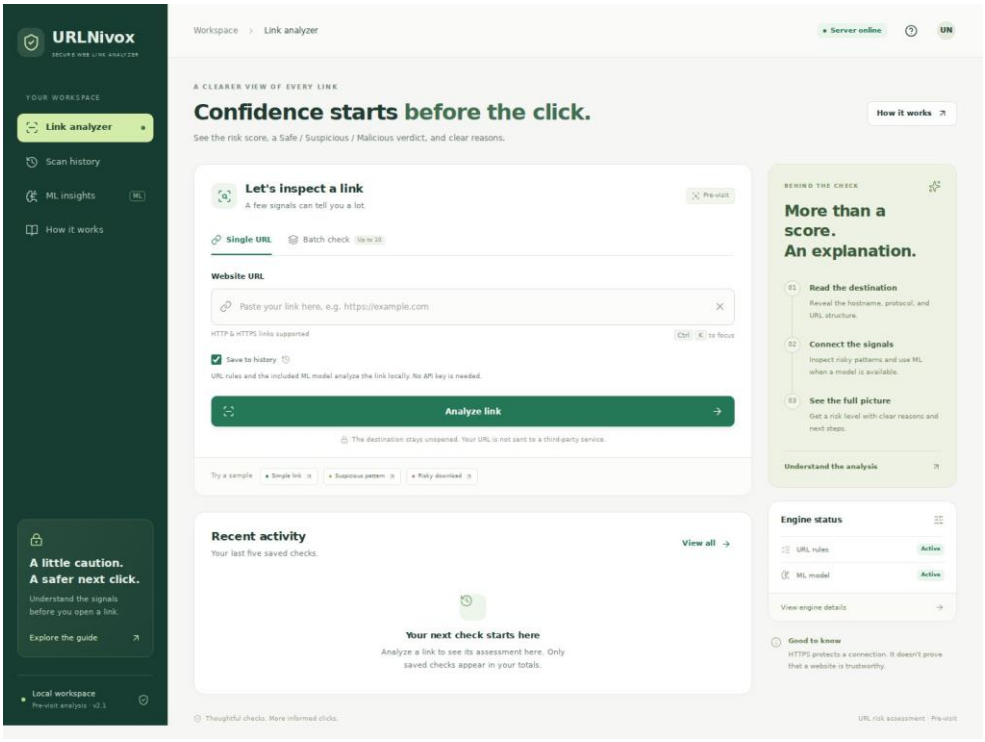


Figure 1.1: URLNivox dashboard from the supplied project preview, showing single-link analysis and the active model.

Understanding the application as a complete workflow

The project has two connected outputs. The first is a prediction produced from a numerical representation of a URL. The second is an assessment that a person can inspect. A model can supply the first output without providing the second. URLNivox connects them through input validation, component scores, explanatory checks, model status, and optional history. This makes the application useful for demonstrating how a machine-learning result becomes part of a working software system. [P1]

Consider a user who receives an unfamiliar account-verification address. The user can enter the address without asking the analyzer to open it. The application identifies the hostname and other URL components, evaluates the observed patterns, and presents an assessment. The result may include a warning about a credential-related term or an executable-style path. These statements explain what the analyzer observed in the text. They do not establish who operates the destination or what a page would do if opened.

This distinction guides the report. The implementation chapters describe observable behavior, while the results chapter describes a particular experiment on labelled historical records. Keeping those two forms of evidence connected allows a reviewer to move from a displayed score to its calculation and from a reported accuracy to its test counts. The application name, URLNivox, identifies the interface; Secure Web Link Analyzer describes the project and its assessment task.

1.2 MOTIVATION

A link often reaches a user with very little context. It may appear in a message, an email, a document, or a social media post. Before opening it, the user can inspect the visible address, but this becomes difficult when the address contains several subdomains, long query parameters, encoded characters, or familiar brand words in unfamiliar positions. The project addresses this inspection stage. It turns URL text into a structured assessment that can be read before any interaction with the destination.

The central design decision is to keep analysis local. A submitted destination is treated as data, rather than as a resource to retrieve. This avoids making a suspicious page part of the normal analysis path. It also makes the limits of the result clear: a URL can contain useful warning signals, but its text cannot reveal every behaviour of the website behind it. The software therefore reports the evidence it actually examines.

Why a visible explanation matters

Long URLs can hide important components among less informative text. A familiar word may appear in a path, query value, or subdomain without being the registered domain. Manually searching for that word does not reveal which role it plays. The analyzer makes these roles explicit through parsing and feature extraction. The practical motivation is to reduce the effort required to inspect the address, while preserving enough detail for the user to question the result.

A single numerical score can conceal two different situations: a low model estimate after successful inference, or a missing model that contributed nothing. The interface must distinguish them. Otherwise, an installation problem could be mistaken for a confident classification. The implemented status fields and score breakdown make this difference visible, which is particularly useful when demonstrating the project on a different computer.

The motivation also concerns repeatability. A saved result can be revisited, and a printed example can be traced to source code. This is more informative than a demonstration that shows only a colored label. The same URL, compatible model artifact, and feature definitions should support an explanation of the same computation, although timestamps and measured request duration can differ. This expectation is checked through the supplied tests and recorded examples. [P1, P3]

1.3 PROBLEM STATEMENT

The problem is to produce a useful and explainable risk assessment from URL text without contacting the submitted destination. The system must accept common HTTP and HTTPS addresses, reject malformed or unsupported input, extract a consistent feature vector, combine rule and model outputs, and present the result in a form a non specialist user can understand. It must also continue working when the optional model cannot be used.

The analysis is a decision support task. A score is an indicator of observed patterns, not a direct measurement of harm. For example, an executable style path can be legitimate, and a short HTTPS address can still point to a compromised site. The implementation should not conceal these ambiguities behind a confident label.

Separating the engineering problem from the prediction problem

The prediction problem is to map URL-derived measurements to the supplied benign or phishing labels. The engineering problem is to make that mapping usable and inspectable within an application. These problems overlap but require different evidence. A confusion matrix describes classification errors. It does not show that malformed JSON is rejected, history can be disabled, or a missing model is reported correctly.

The report therefore treats a successful analysis as a sequence of conditions. The address must first satisfy the input policy. Its numerical features must follow the model schema. The appropriate inference or fallback path must run. The component scores must be combined according to the documented formula. Finally, the browser must show the returned information without treating the submitted address as a navigation command.

Failure at an earlier condition should not be disguised by a later score. An invalid port is a validation error, rather than a harmless address. A storage problem is a persistence warning, rather than a failed classification. A model mismatch is an availability condition, rather than evidence that a URL has low risk. This separation makes both the implementation and its assessment easier to review. [P1]

1.4 OBJECTIVES

1. Validate and normalize submitted HTTP or HTTPS URLs using an explicit, testable policy.
2. Extract the same 24 numerical features during training and inference.
3. Produce separate rule, structure, and model outputs, followed by a transparent hybrid score.
4. Support single URL analysis and batches of up to ten links.
5. Provide optional local history with search, filtering, pagination, report reopening, and deletion.
6. Protect common secret values in displayed and stored URLs and expose model readiness honestly.
7. Record dataset provenance, split information, actual classifier metrics, and reproducible tests.

Relating objectives to review evidence

Each objective needs an observable completion condition. For input handling, the evidence is an accepted normalized representation or a structured validation error. For feature consistency, it is the shared ordered feature list used by training and serving. For model integration, it is a loaded artifact together with an actual prediction and the expected contribution to the final score. The project does not rely on a screenshot alone to establish these conditions.

Explanation is reviewed by checking that the result exposes the direct rule score, structural score, heuristic score, model score, and final score. Persistence is reviewed by comparing saved and private requests. Reproducibility is reviewed through the dataset identity, split metadata, estimator settings, and a test that recomputes the stored measurements. These checks give the objectives practical meaning without assigning an unsupported success percentage to the application as a whole.

There is also a difference between a completed objective and an open research question. The software can successfully combine rules and a classifier even if the combined score has not been shown to improve predictive performance. The integration objective is supported by code and arithmetic checks. A claim of improved detection would require a separate comparison using the same held-out records, decision rules, and evaluation criteria. [P1, P2, P3]

1.5 SCOPE OF THE PROJECT

The implemented input is URL text. The backend examines its syntax, hostname, path, query structure, selected terms, and numerical characteristics. The trained model receives only the documented feature vector. There is no page content classifier, screenshot analysis, live reputation service, domain age lookup, WHOIS query, certificate inspection, or redirect expansion in the runtime pipeline.

Dataset preparation is a separate maintenance operation. It may download one pinned public dataset file, but it does not open the addresses contained in that file. Keeping this distinction explicit prevents the phrase “offline analysis” from being misunderstood as a claim that installation or dataset acquisition never requires a network connection.

The intended setting is a local demonstration or a personal inspection workflow. A user pastes a link, reviews its components and warnings, and decides whether to investigate further using a trusted route. The project is not an antivirus engine, a browser enforcement mechanism, or a certification service. This scope informs both the implementation and the interpretation of the evaluation results [P1, P2].

Evidence boundary

Table 1.1: Implemented claims and their evidence boundaries.

Statement supported by the project	Evidence boundary
The included ML model is active	Runtime model integration tests and example results
The held out ML accuracy is 89.96%	Historical binary classifier evaluation only
The analysis path does not fetch destinations	Source design and mocked network tests
Saved reports redact recognised secrets	Implemented redaction rules and tests
The prototype supports three final verdict bands	Implemented thresholds and demonstration inputs

The project does not establish a production detection rate, a guaranteed safety verdict, or a measured improvement over competing systems. It also does not establish that the 60 to 40 hybrid weighting is optimal. Those questions require separate experiments with a defined deployment population and a locked evaluation protocol.

Meaning of local operation

Local operation refers to the destination-analysis path. The browser communicates with the Flask service, and that service evaluates the submitted URL using its installed code and model. The design does not require the destination to be online because the assessment uses the address text. An unavailable website and an available website with the same URL representation would therefore receive the same feature-based treatment from this path.

This does not mean that every activity in the project is disconnected from the internet. Installing dependencies and acquiring the pinned dataset are separate preparation activities. The dataset preparation script can retrieve its documented source file, but it does not open the destinations listed in that file. Separating preparation from inference keeps the user's analysis request independent of those maintenance operations.

The application also has a local storage boundary. Optional history writes to SQLite on the machine running the service. A private request disables that history write; it does not erase text from the user's clipboard, browser memory, screenshots, or an export the user deliberately creates. Describing these boundaries precisely avoids turning a specific implementation property into a broader claim that the project does not establish. [P1]

1.6 CONTRIBUTIONS

The first contribution is an integrated workflow. URL parsing, feature extraction, heuristic scoring, model inference, report generation, and optional persistence are connected through a small Flask application. The user does not need to run a separate training command for each check because a trained model is supplied with the project.

The second contribution is consistency between training and serving. Feature names are declared once in a fixed order. The model artifact records that order, and the runtime verifies it before making predictions. This addresses a practical source of silent errors: passing valid numbers to a model in the wrong column order.

The third contribution is the separation of evidence. Structural heuristics are labelled as heuristics, and the ML component is identified as a trained Random Forest. The final score records the weights used. When the model is missing, incompatible, or unable to predict, the result explicitly switches to the heuristic path instead of fabricating a probability.

The fourth contribution is traceability. The dataset metadata preserves the upstream source, pinned revision, checksums, class mapping, preparation seed, cleaning counts, and source row identifiers. The evaluation report records the group split and confusion matrix. Together, these records allow an examiner to connect the printed claims to the supplied files.

Contribution to an examinable project

The main contribution is the connection between artifacts that are often presented separately: a web interface, an analyzer, a training procedure, an evaluation file, and tests. The project makes those connections explicit. The feature tuple identifies what enters the classifier, the model metadata identifies the associated experiment, and the report explains how the resulting probability enters the final assessment.

Another contribution is graceful handling of incomplete operating conditions. The absence of an ML artifact does not prevent the rule engine from producing an explanation. At the same time, the application does not pretend that an unavailable model supplied a prediction. This behavior provides a practical demonstration of reliability: useful work continues, but the change in evidence remains visible to the user.

The contribution should be assessed at the scale supported by the project. It is a reproducible prototype, rather than a new dataset collection or a proven replacement for existing security products. Its value for academic review comes from the ability to inspect the pipeline, reproduce recorded results, and identify concrete next experiments. These are engineering outcomes that can be demonstrated with the supplied materials. [P1, P2, P3]

1.7 ORGANISATION OF THE REPORT

Chapter 2 places the project in the context of URL analysis research. Chapter 3 explains the system design, requirements, modules, database, and API. Chapter 4 documents preprocessing, feature extraction, the trained model, selected code, input, output, and score calculations. Chapter 5 presents the measured results and test evidence. Chapter 6 reviews the objectives and identifies limitations and future work. References and a reproducibility appendix complete the report.

Reading paths for implementation and evaluation

A reader reviewing implementation can begin with the architecture and module responsibilities in Chapter 3, then follow the same request through Chapter 4. The feature tables define the numerical inputs, the rule tables define the heuristic points, and the calculation section connects those components to the final verdict. The short code excerpts identify the source locations for inspecting that flow in the complete project.

A reader reviewing the experiment can begin with dataset preparation and training in Chapter 4, then use Chapter 5 to connect the held-out records to the confusion matrix and metrics. The reproduction instructions in Appendix A explain which artifacts and commands support a numerical check. This reading path keeps model training, evaluation, and application scoring connected without treating them as the same operation.

Numbered references identify external research and software documentation. References prefixed with P identify the supplied project materials and verification evidence. The figure and table lists provide direct page references for the architecture, feature definitions, score rules, recorded outputs, and results. Proposed experiments are collected in Chapter 6 so that a reviewer can distinguish the completed work from the next development steps.

CHAPTER 2

LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

The survey by Sahoo, Liu, and Hoi organizes malicious URL detection around feature representation, learning methods, and practical deployment challenges [1]. Its relevance to this project is the distinction between information available directly in the URL and information obtained from hosts or page contents. URLNivox deliberately restricts its runtime inputs to the first category.

This restriction gives the project a clear engineering boundary. Lengths, character ratios, hostname patterns, and selected terms can be computed without opening a destination. Their usefulness depends on whether patterns in labelled historical data remain relevant to later inputs. The report therefore treats generalization as an evaluation question rather than as a consequence of using machine learning.

Using the literature to define comparable tasks

The survey by Sahoo and colleagues provides a useful starting point because malicious-URL assessment can involve several kinds of evidence and learning approaches. Its role here is to organize the problem, rather than to supply an accuracy target that this project must match. URLNivox selects URL-derived evidence for its implemented path and states that choice explicitly. [1]

Before comparing two systems, the reviewer should ask what each system observes. A method that examines downloaded page content has information that this implementation does not collect. A reputation lookup answers a question about a maintained external source. A lexical classifier learns from address-derived measurements. The methods may share the word detection while using different inputs, labels, and operating assumptions.

The same care applies to experimental results. A percentage obtained on one dataset cannot be treated as a direct ranking against a percentage obtained on another. The class distribution, time period, split procedure, and unit of separation may differ. This report therefore uses the literature to justify design questions and uses the supplied project evidence to describe the implemented result. It does not merge published experiments with the project's confusion matrix.

A useful reading approach is to record three items for each source: the representation, the evaluation setting, and the conclusion that the source actually supports. This

prevents a broad research claim from being transferred to a narrower prototype. It also makes the remaining questions easier to state, such as whether URL-only patterns generalize to a later collection or whether an added component improves a particular error tradeoff.

2.2 LEARNED URL REPRESENTATIONS

Le, Pham, Sahoo, and Hoi propose URLNet, which learns URL representations using character and word information with convolutional neural networks [2]. The work illustrates an alternative to manually selected lexical features. URLNivox uses a smaller, explicitly defined numerical representation because the project places emphasis on inspectable inputs, simple local inference, and a manageable implementation.

This is a design comparison, not a measured performance comparison. URLNet and the supplied Random Forest have not been evaluated on the same data partitions in this project. No claim is made that the simpler representation is more accurate. A fair comparison would require common labels, equivalent split rules, a shared evaluation set, and recorded computational conditions.

Hand-defined and learned representations

URLNet learns representations from character and word information using a deep-learning approach. That research demonstrates a different way to represent URL strings from the fixed numerical measurements used in this project. URLNivox does not implement URLNet, reuse its trained parameters, or claim its experimental results. [2]

For this implementation, the hand-defined representation creates a small, inspectable interface between data preparation and inference. Each input has a name and a stated meaning. A reviewer can inspect whether a URL has an IP literal, how its query length is measured, or how many configured terms are found. This supports debugging and a clear project explanation, although the chosen measurements cannot describe every detail of an arbitrary address.

Representation changes would affect more than the training script. Replacing the feature vector with a character sequence, for example, would require a different model interface, preprocessing rules, artifact checks, and explanation strategy. The existing feature-name comparison would no longer be sufficient. A fair experiment would preserve the same evaluation split while documenting the new representation and its training procedure.

The important comparison is therefore a design tradeoff to investigate, not an assumed winner. Fixed features make the current calculation easy to inspect. A learned

representation offers another research direction. Choosing between them for a future version should depend on an experiment that measures the required outcomes under the same conditions, together with the practical cost of maintaining and explaining the resulting system.

2.3 PUBLISHED DATASET FOUNDATION

The DEPHIDES repository by Buber, Sahingoz, and Kugu provides labelled phishing and legitimate URL data and a deep learning implementation [3]. Its documentation identifies PhishTank and Common Crawl as the collection sources. URLNivox uses a balanced sample from a pinned file in that repository, while retaining the supplied labels and recording the transformation steps.

The current project does not reproduce the original DEPHIDES experiment. It creates its own domain grouped split from the selected sample. Results from the two projects should therefore be described in their own experimental settings rather than placed in a single ranking without qualification.

Dataset identity as part of the method

A dataset name alone is not enough to identify the project's training input. The same repository can contain multiple files or later revisions. This project records the source commit, source-file checksum, transformation procedure, selected class counts, and prepared CSV checksum. Those details identify a specific artifact and make it possible to distinguish an exact reproduction from a new preparation run. [P2]

There are two stages of interpretation. The upstream data provides labels for historical URL records. The project then validates and samples those records to construct its own training input. It preserves the author-provided class meanings rather than deciding labels by opening websites. As a result, a record's label belongs to the documented dataset setting and is not a current verdict about the destination.

The source-row field supports a useful audit. A reviewer can connect a prepared example to its position in the pinned upstream file and inspect how it entered the sample. This is different from manually selecting only persuasive examples for a demonstration. The three addresses shown in the application output are illustrative execution examples, while the 40,000-row CSV supplies the labelled experiment.

Keeping these roles separate also helps future work. A later dataset should be stored with its own identity and collection description. Its results can then be compared openly with the historical experiment. Silently replacing the existing CSV while continuing to quote the old metrics would break the connection between the report and the evidence.

2.4 FAMILIES OF URL ASSESSMENT METHODS

Different detection approaches observe different parts of a link's behaviour. This makes their strengths and limitations dependent on the task. The table describes design characteristics rather than a benchmark ranking. The comparison is informed by the research background [1, 2] and by the implemented boundary of URLNivox [P1].

Table 2.1: Comparison of URL assessment approaches.

Approach	Main input	Practical limitation	Use in this project
Static URL rules	Explicit properties of the address	Rules can miss new patterns and flag legitimate unusual links	Implemented with visible reasons
Lexical ML model	Numerical features from URL text	Depends on training labels and feature coverage	Implemented as Random Forest
Learned string model	Character or token representation	Requires a different training and model serving workflow	Future comparison candidate
Reputation list	Previously recorded destination verdicts	Coverage and freshness depend on the provider	Not queried at runtime
Host information	DNS, registration, or certificate evidence	Requires additional data and may involve external lookups	Outside current scope
Page content analysis	Retrieved HTML, scripts, or visual content	Introduces retrieval, isolation, and content handling requirements	Outside current scope

Comparing information requirements

The families in the comparison table answer different questions. Manual inspection helps a person understand visible structure. A static rule list applies explicit conditions. A learned classifier uses a fitted relationship between measurements and labels. A live reputation service adds information maintained outside the application. These categories can be combined, but their contributions should remain distinguishable in both the design and the reported result.

In URLNivox, the runtime contract fixes the available information before the score is calculated. No later stage silently adds page content, a certificate check, or a DNS answer. Consequently, the explanation can be traced back to URL features and the local artifact. The design has a clear evidence boundary, while accepting that some threats and legitimate behaviors cannot be resolved from that boundary alone.

An extension involving another source would need an explicit policy for unavailable or conflicting information. For example, a future reputation component might have no record for a URL. That absence would need its own representation; treating it automatically as a benign result would change the meaning of the score. Similar questions arise for stale records, lookup errors, and inconsistent labels.

This comparison leads to a practical requirement: identify the origin of each displayed claim. A feature observation, a rule warning, a model prediction, and a stored historical report should not be presented as if they were the same kind of evidence. The current response structure supports this separation through distinct fields and component scores. [P1]

2.5 RULES AND MACHINE LEARNING

Rules are useful when an observation should be directly explainable. For example, credentials before the hostname can be highlighted without requiring a learned model to assign a large weight to that feature. A classifier can combine several weaker measurements in ways that are not captured by a short list of manually chosen conditions.

Combining these components does not automatically improve accuracy. A rule can add noise, and a trained model can inherit biases from the data. The implemented weighting is therefore presented as a project design choice. The report evaluates the supplied binary classifier and checks the hybrid arithmetic, but it does not invent an ablation result for the combined system.

Agreement does not prove independent evidence

The rule engine and the classifier both inspect information derived from the same URL. Their agreement is useful to display, but it should not be described as agreement between two independent observations of a website. A keyword can influence an explicit rule and also contribute through a learned feature. This overlap is a reason to expose the calculation instead of presenting the components as independent confirmations.

The weights express the implemented policy for combining those components. They do not measure how much each component contributes to predictive accuracy. A 40% model weight means that the model score receives that multiplier in the arithmetic. It does not mean that the model correctly resolves 40% of attacks or that the rules explain the remaining 60%.

A component comparison would require an evaluation plan. The reviewer could hold the test partition fixed and evaluate the model alone, the heuristic engine alone, and the combined decision. The output definitions and thresholds would need to be made comparable before calculating differences. Without that experiment, the defensible conclusion is that the components are integrated and the arithmetic is reproducible, not that the selected weights are optimal.

This distinction also clarifies explanation. The rules provide descriptions of triggered conditions. They do not reveal the exact contribution of every forest feature for a

particular prediction. The model's global importance values answer another question and are discussed separately in Chapter 5. [P1, P2]

2.6 SOFTWARE FOUNDATION

The project uses scikit-learn for its estimator and evaluation functions [4]. Flask provides the local web interface to the Python backend. SQLite supplies a file based history store through Python's standard database interface. These choices keep the components small enough to inspect and reproduce in a student project.

Library behavior and project-specific behavior

Established libraries supply parsing, web serving, storage access, learning, and numerical operations. The project still defines how those operations are combined. For example, a parsing function can separate URL components, but the analyzer chooses which schemes and inputs to accept. The Random Forest implementation can estimate class probabilities, but the project chooses its feature vector, training data, and displayed score bands.

This division makes source attribution more precise. Documentation is used to understand the library's interface and behavior. The project files are used to establish the actual parameters and control flow. The existence of a library feature does not imply that the application uses it. In particular, a capability described in scikit-learn should not be listed as an implemented experiment unless it appears in the project's training or evaluation path.

Version recording supports the same distinction. The stored runtime metadata describes the environment associated with the supplied model. It helps a reviewer reproduce that artifact and interpret compatibility warnings. Installing a different package version may still allow some code to run, but it is not automatically an exact reproduction of the documented experiment. The report therefore retains the recorded versions alongside the model settings. [P1, P2, 9]

2.7 LIMITATIONS OF EXISTING WORK AND PROJECT GAP

The practical gap addressed here is the distance between a prediction script and an understandable application. A classifier may output a class or probability, but a usable inspection tool also needs input validation, consistent features, model status reporting, privacy handling, error messages, stored report retrieval, and reproducible evidence. These concerns are central to the implemented system.

The project does not claim to introduce a new machine learning algorithm. Its contribution is the assembly and verification of a local analysis workflow with explicit boundaries. A reviewer can inspect the rule points, feature order, stored artifact,

dataset metadata, and API response instead of relying on an unexplained claim of intelligent detection.

Table 2.2: Project design decisions and their consequences.

Decision	Reason	Consequence
URL text only	Preserve a pre visit runtime boundary	Destination content remains unknown
Fixed numerical feature order	Keep training and inference consistent	Schema changes require retraining
Random Forest with 350 trees	Use an inspectable, supplied estimator configuration	Artifact and dependency versions must be retained
Optional local history	Support review without requiring every check to be stored	User must choose private checks when appropriate
SQLite storage	Keep installation simple	Shared multiuser operation is not implemented
Heuristic fallback	Keep analysis available when ML fails	Engine status must accompany the score
Domain grouped split	Reduce overlap of related URL groups	Domain approximation remains a limitation

Unresolved evaluation questions

Three questions remain open. First, does the combined score improve useful detection when compared with rules and ML separately on a common evaluation set? Second, how much of the classifier's performance depends on collection differences such as the presence of HTTPS? Third, how well does the model behave on later URLs from independent sources?

These questions guide the proposed future experiments. They are not treated as completed work. The appropriate next step is to define a validation protocol, reserve a final test set, and compare alternatives without repeatedly tuning against the same held out records.

Turning gaps into answerable questions

A project gap becomes useful when it can be expressed as a question with a defined observation. The present implementation addresses whether a local URL classifier can be incorporated into an explainable browser workflow with explicit fallback and reproducible evidence. The source, stored model, and tests provide observations for that question. They do not answer every broader question about malicious websites.

The next research questions can be separated by what they would change. A later-data experiment tests transfer across time. A different-source experiment tests dependence

on collection conditions. A component ablation tests the effect of removing one part of the system. A calibration study asks how predicted probabilities correspond to observed frequencies under a stated setting. Each question needs a separate result rather than another interpretation of the same accuracy figure.

This approach avoids using a long list of possible enhancements as a substitute for an evaluation plan. A proposed change should identify the input artifact, the unchanged comparison condition, the metric, and the decision it will inform. If a new parser changes domain grouping, for example, the resulting partition should be documented because it may change the experiment as well as the implementation.

For the current report, the unresolved questions are retained as future work. This preserves a clear boundary between completed engineering and proposed investigation, while giving the project a concrete path for further development.

CHAPTER 3

SYSTEM DESIGN

3.1 EXISTING SYSTEM

A user can inspect an address manually, consult a reputation list, or submit it to a classifier. These approaches provide different kinds of evidence. Manual inspection exposes obvious spelling and structure changes but becomes difficult for long addresses. A reputation result concerns what a source already knows. A standalone prediction script usually returns a label without providing the input handling, explanation, model readiness, and history needed by an end user [1, 2].

The project takes these methods as its design context. It does not depend on a live reputation account and does not present a prediction as a direct observation of website behaviour. The existing approach being extended is a local numerical classifier: URLNivox places that classifier within an inspectable application workflow.

Baseline workflow considered in the design

The design begins with a simple baseline: a user supplies an address and receives a model label. This baseline is easy to describe, but it leaves several surrounding operations unspecified. Someone must prepare a valid feature vector, ensure that the correct artifact is loaded, decide how errors are shown, and explain what the label means. These responsibilities remain necessary even when the prediction function itself is short.

In an academic demonstration, a baseline script can be useful for examining the model in isolation. In a browser application, the same script needs an input contract and a response contract. The user should not have to inspect a Python exception to discover that a port is invalid or a dependency is missing. The design therefore treats request handling and model inference as separate responsibilities with explicit connections.

The baseline comparison is conceptual. No user study or timed comparison with competing products was conducted for this report. The purpose is to identify the additional behavior implemented around the classifier and to provide a reference point for the requirements. Any later comparison of usability or efficiency would need its own participants, tasks, measurements, and results. [P1, P3]

3.2 DRAWBACKS OF THE EXISTING SYSTEM

A label alone does not tell a user whether the model loaded successfully, which URL was actually parsed, whether secret query values were stored, or which features supported a warning. These omissions can produce a misleading demonstration even when the classifier itself works. A separate problem arises when training and serving disagree about feature order: the program may accept the input while passing semantically incorrect columns to the model.

The proposed design addresses those engineering weaknesses through shared feature definitions, explicit model status, a score breakdown, validation errors, and optional local history. It does not remove the statistical limits of historical training data; those limits remain part of the interpretation of every result.

Failure modes that shape the requirements

One important failure mode is silent mismatch. A vector can contain the right number of values while placing them in the wrong order. The estimator may still produce a numerical answer, even though the values no longer represent the features it learned. The shared tuple and artifact feature-name check address this risk by making the schema an explicit contract.

Another failure mode concerns ambiguous success. An analysis can succeed while a history write fails, or the server can be online while the model is unavailable. A single generic success indicator cannot communicate these differences. The response therefore includes analysis status, model status, and persistence information separately. The browser can use those fields to describe what completed and what did not.

A third failure mode arises when a displayed report is confused with current analysis. A stored report represents a previous execution using the model and features available at that time. Reopening it should not silently imply that the latest artifact has reevaluated the address. The archived indicator and stored JSON support this distinction. These design choices make errors and state changes visible rather than trying to eliminate every possible operating failure. [P1]

3.3 PROPOSED SYSTEM

The primary user is a person who wants to inspect a URL before opening it. The secondary user is a student, developer, or examiner who needs to understand how the result was produced. The system therefore has two related outputs: a readable assessment for ordinary use and structured evidence for technical review.

The supported deployment is a local Python process serving its own browser interface. The backend binds to 127.0.0.1 by default, uses port 5000 unless configured otherwise, and runs with debugging disabled. The current program does not contain accounts, roles, tenant separation, or a public hosting configuration. These are meaningful constraints rather than missing details to be assumed away.

Normal checks do not require an external threat intelligence account or paid API. Initial dependency installation may need internet access. The supplied dataset and model allow a demonstration without rerunning data acquisition or training. The history database is created when needed, and an environment variable can redirect it to a separate demonstration file.

The application is suitable for a supervised demonstration because its data flow can be explained and reproduced. Public or shared deployment would require additional engineering, including authentication, request origin protections, a suitable WSGI server, deployment monitoring, and a revised storage policy. These additions are proposed work, not properties of the current prototype [11].

Responsibilities of the user and the application

The application is responsible for accepting a valid request, applying its documented checks, and returning an understandable result. The user remains responsible for deciding how to act on that result. The interface supports inspection by showing the parsed destination and component evidence. It does not automatically open the destination, submit credentials, or complete any action described by the link.

Two ordinary use paths illustrate the design. In a private check, the user disables history saving, submits an address, and reviews the returned assessment. In a saved check, the analysis is also recorded locally and can later be found through history. The underlying scoring procedure is the same; persistence changes the record lifecycle, not the model input or weighting formula.

The administrator role in the prototype is the person maintaining the local project files. That person installs compatible dependencies, retains a trusted model artifact, and decides whether to run preparation or training commands. There is no implemented web account system separating administrator and ordinary user permissions. The report therefore describes a local demonstration setting and identifies shared deployment as a separate design problem. [P1]

3.4 SYSTEM ARCHITECTURE

URLNivox uses a browser client and a local Flask process. The same process serves the interface assets and the JSON API. Within the backend, URL analysis is separate from HTTP request handling and database access. This structure allows the analyzer to be exercised directly in tests without starting a browser.

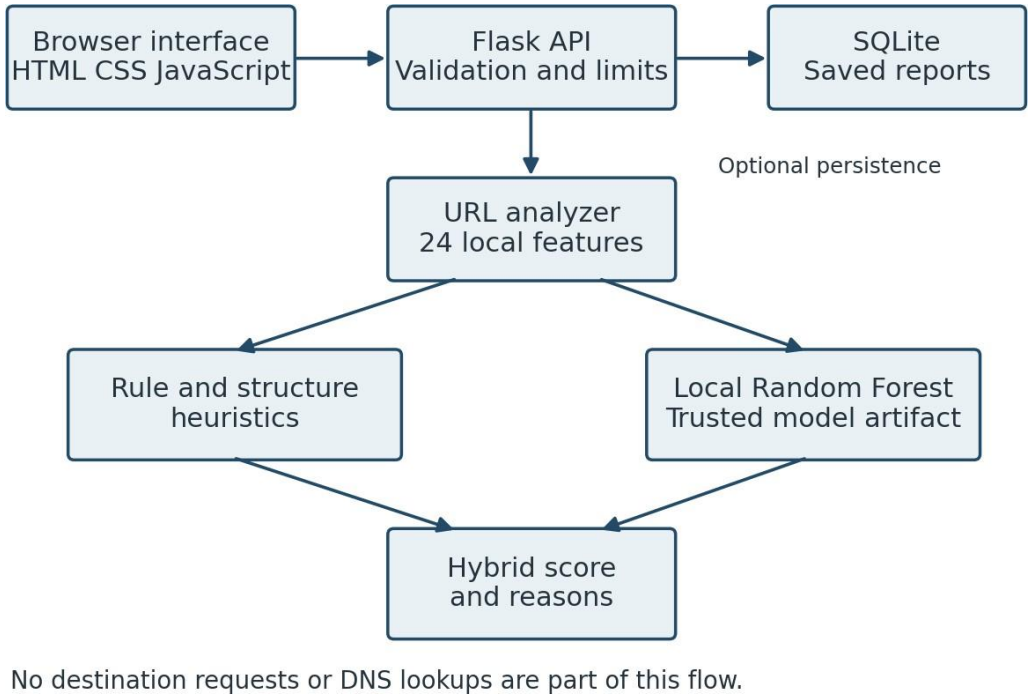


Figure 3.1: Application architecture and local dependencies

The analyzer constructs one feature dictionary and sends the relevant values to the rule, structure, and model components. The final report preserves their separate outputs. SQLite receives a redacted report only when saving is enabled. The model artifact is loaded from the local backend directory and is reused through a cache when its file signature remains unchanged.

There is no destination website component in the runtime call graph. This is a deliberate property of the implementation, not a missing arrow in the diagram. Model training and dataset acquisition are maintenance tasks outside a user's analysis request. The source code separates these tasks into their own executable scripts [P1].

Runtime and maintenance paths

The architecture contains a runtime path and a maintenance path. The runtime path begins with a browser request and ends with a report, optionally followed by a database write. It uses the installed model for inference. The maintenance path begins with a dataset source and ends with a prepared CSV, a trained model, and evaluation metadata. Keeping these paths separate prevents ordinary URL checks from unexpectedly changing the classifier.

The source modules reflect this separation. The analyzer and model adapter cooperate during assessment. Preparation and training import the shared parsing and feature logic, but they are executed as explicit commands. This reuse is important: training should not learn from a different interpretation of URL length or root paths than the one used when the browser submits a new example.

The database is an output-side component. It stores an assessment for later inspection; it does not supply extra evidence to the current prediction. A previously saved URL is not automatically trusted or blocked because it appears in history. Likewise, the browser's recent-check list is a view of stored results, not an external reputation feed.

These relationships make the architecture suitable for incremental review. A reviewer can inspect the analyzer without starting the browser, test the API with a temporary database, and reproduce model metrics using the supplied data. Integration then checks that these parts exchange the expected fields. [P1, P3]

3.5 ARCHITECTURAL FLOW

The browser sends user supplied text to a local API endpoint. The request handler checks the payload shape and analysis limits before calling the analyzer. The analyzer validates the URL, extracts features, calculates component scores, obtains the optional model output, and constructs a redacted display URL. The application then either saves the report or returns it as a private check.

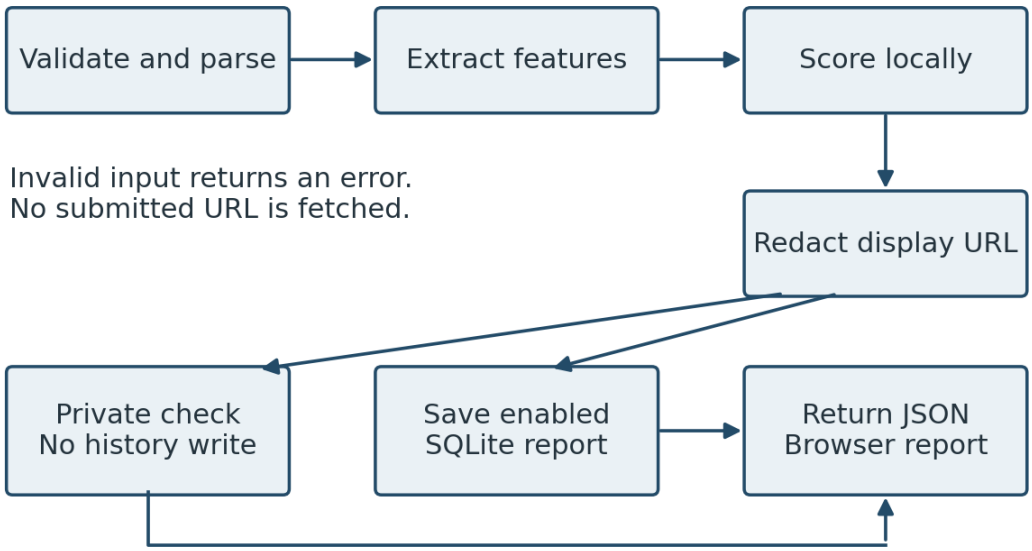


Figure 3.2: Analysis flow with optional local persistence

URL text is untrusted input. It is parsed and validated, but it is not executed or fetched. Browser output must also treat it as text. The front end includes HTML escaping before inserting variable content into generated markup. SQL queries use parameters for submitted values and history filters.

The model artifact has a different trust requirement. It is a locally supplied joblib file. The runtime verifies its expected structure after loading, but these checks do not make an arbitrary pickle based artifact safe. A replacement model must come from a trusted source or be produced by the supplied training procedure [9].

The SQLite database is local application data. It is not a multiuser access control boundary. Anyone who can access the local service can use its history endpoints. This constraint is relevant when deciding whether the prototype can be exposed beyond the local machine.

Following one request through the data flow

At the entry point, the API distinguishes a request object from the URL string inside it. Payload validation establishes the expected JSON structure and the type of the history preference. URL validation then establishes whether the address can enter the analysis pipeline. These checks occur at different layers because a syntactically valid JSON object can still contain an unacceptable URL.

After validation, the analyzer creates observations and numerical features. The rules and structural checks use those observations to create component scores and explanations. The model adapter converts the ordered feature vector into an optional model result. The analyzer then combines the available components and constructs the

display-safe URL. This order explains why redaction must preserve useful structural indicators while avoiding the return of recognized secrets.

The API adds operational fields around the completed assessment, including duration and history status. If saving is requested, it attempts to store the report. A storage error changes the persistence fields but does not require the classifier to run again. The result sent to the browser therefore contains both analytical information and information about the request's completion.

For batch mode, this flow repeats for each address after the outer list has been validated. The index field connects every result to its input position. This makes mixed outcomes understandable: a batch can contain successful assessments and validation errors without losing their order. [P1]

3.6 MODULE DESCRIPTION

Table 3.1: Principal Python modules and responsibilities.

Module	Responsibility	Main interface
analyzer.py	URL validation, features, rules, redaction, and final score	analyze_url
feature_extractor.py	Fixed ordered model input schema	to_ml_feature_vector
ml_model.py	Artifact validation, caching, and inference	predict_url_risk
app.py	HTTP routes, request limits, and history	Flask application
train_model.py	Grouped split, model fitting, and metrics	Training command
prepare_dataset.py	Pinned source preparation and provenance	Dataset command

Individual analysis

The user opens the local dashboard, selects the individual mode, and enters a URL. The save setting determines whether the result will enter history. The browser sends the request to the local Flask service. If input validation fails, the interface shows the returned explanation. Otherwise, it opens a report containing the final score and its supporting components.

The user can inspect the detected hostname, rule checks, structure checks, and all 24 feature values. Model availability is visible. The user can export the report as JSON, copy a text summary, or use the browser's print workflow. These operations concern the assessment and do not navigate to the submitted destination.

Batch analysis

Batch mode accepts one address per line. The interface displays the number of entries against the limit of ten. A batch result contains a success or error for each original position. Valid results can be opened individually, and the displayed batch can be exported as CSV. One invalid entry does not erase the valid reports from the same batch.

History and model review

Saved checks appear in the local history view. Search operates on stored URL text, and a verdict filter narrows the results. Pagination prevents the interface from loading the entire history at once. Opening a modern record returns its stored JSON assessment with an archived indicator. Older rows without report JSON remain accessible as legacy summaries.

Table 3.2: Principal application use cases.

Use case	Preconditions	Main result
Analyze one link	Local service is running	Current assessment or validation error
Analyze a batch	One to ten text entries	Per item results with success count
Reopen a report	A saved record exists	Archived report or legacy summary
Use private mode	Save setting is disabled before submission	Result is returned without insertion
Clear history	User confirms the interface action	Saved rows are deleted locally

The model insights page is a separate review use case. It shows readiness, dataset size, split information, training date, and matching metrics. If the model or metrics are unavailable, the interface reports that state rather than substituting estimated values.

Database design

The application uses one SQLite table, url_analysis, through Python's sqlite3 interface [8]. A summary supports listing and filtering, while report_json preserves the detailed result. The schema is created when needed. If an older table lacks report_json, the application adds that column without deleting earlier records [P1].

Table 3.3: History database fields and their meanings.

Field	SQLite type	Meaning
id	INTEGER PRIMARY KEY	Automatically assigned record identifier
url	TEXT NOT NULL	Redacted display URL
verdict	TEXT NOT NULL	Safe, Suspicious, or Malicious
risk_score	INTEGER NOT NULL	Final score shown in the report
rule_score	INTEGER NOT NULL	Sum of direct rule points after capping
ml_score	INTEGER NOT NULL	Legacy column containing the structure score
reasons	TEXT	Joined text reasons for history summaries
created_at	TIMESTAMP	Database insertion time, default current timestamp
report_json	TEXT	Serialized detailed assessment

The legacy column name ml_score is misleading in the current schema. The insertion code stores signal_based.signal_score in that column. The actual Random Forest score is stored in report_json under ml.risk_score. Analysis of database exports must respect this distinction. A future migration could rename the legacy column while preserving compatibility.

Record lifecycle

Saving is enabled by default unless save_history is false. The application inserts the redacted URL, summary values, reasons, and full report JSON. The returned response adds the newly assigned history identifier. Reopening a record restores its saved assessment and adds an archived flag; it does not rerun the model on the URL.

History listings order records by descending identifier. Search uses a parameterized, case insensitive literal substring condition, and the verdict filter accepts only known labels. A delete request clears the table and returns the removed row count. The browser asks for confirmation before making that request. No automatic retention period or database encryption is implemented.

If a history write fails, the analysis is still returned with a warning and saved_to_history set to false. This keeps the assessment useful while accurately describing the persistence outcome.

API contracts

Table 3.4: Local API endpoints and response contracts.

Method and route	Purpose	Important behaviour
GET /	Serve the browser application	Frontend and API share the local origin
GET /api/health	Service and model readiness	Includes mode, batch limit, and matching metrics
POST /api/analyze	Analyze one URL	Requires a JSON object and string URL
POST /api/analyze/batch	Analyze up to ten URLs	Preserves per item success or error
GET /api/history	List saved summaries	Supports limit, offset, q, and verdict
GET /api/history/id	Reopen one saved record	Returns detailed or legacy content; 404 if missing
DELETE /api/history	Clear saved checks	Returns the deleted row count

Errors and browser policy

Invalid payloads or URLs return a structured 400 response. Oversized bodies receive 413, rate limited analysis requests receive 429, and a missing history identifier receives 404. The application supplies no store caching on API responses, a same origin Content Security Policy, frame blocking, content type protection, and a no referrer policy.

These headers support the intended local browser workflow. They do not add user authentication or make public exposure an implemented feature. The service version reported by the current health endpoint is 2.1.1 [P1].

Data ownership across modules

Each module owns a particular kind of decision. The analyzer owns URL interpretation and scoring policy. The feature adapter owns the numerical order. The model module owns readiness and inference. The web module owns HTTP behavior and optional persistence. Preparation owns source sampling, while training owns fitting and evaluation. This division reduces the chance that the same policy is implemented differently in several places.

The browser is responsible for presenting returned values and controlling interaction state. It should not recompute a competing risk score from visible text. If the browser and backend used separate formulas, a server result and a displayed result could disagree. The current response includes the score breakdown so the interface can explain the backend's computation directly.

Storage introduces a longer-lived data contract. A report may remain in history after the application is updated. The current implementation keeps detailed JSON for modern records and returns a legacy summary when older rows lack that field. This supports inspection of earlier data without inventing fields that were never stored. A future schema change should preserve the same principle: distinguish known historical values from information produced by a newer execution.

Module ownership is also useful during diagnosis. A malformed-address problem belongs first to parsing and validation. A feature mismatch belongs to the shared schema and artifact. A missing report after a completed analysis belongs to persistence. Identifying the responsible layer helps avoid unnecessary changes to the classifier when the issue is elsewhere. [P1]

3.7 DESIGN PRINCIPLES AND REQUIREMENTS

Each requirement below maps to an observable action. This makes acceptance less dependent on visual appearance or a single successful demonstration. The identifiers are used for this report's traceability and are not names of additional source modules.

Table 3.5: Functional requirements and observable acceptance evidence.

ID	Requirement	Acceptance evidence
F01	Accept a valid HTTP or HTTPS URL	JSON report contains normalized display URL and score
F02	Reject malformed and unsupported input	Validation error without destination access
F03	Extract 24 model inputs	Runtime feature order matches artifact metadata
F04	Explain heuristic warnings	Individual checks include titles, details, and points
F05	Use the supplied classifier	Model availability and weighted score are visible
F06	Continue when ML is unavailable	Heuristic engine and unavailable status are reported
F07	Process one to ten URLs per batch	Each item returns its own success or error result
F08	Permit a private check	save_history false produces no history insertion
F09	Search and reopen saved reports	Filtered results and archived detail are returned
F10	Clear local history	Delete endpoint reports the number of removed rows
F11	Export displayed information	Browser offers report JSON and relevant CSV exports
F12	Report real model evidence	Health response includes matching metrics when available

A single check requires a JSON object with a string URL. The optional save_history value must be a Boolean. The batch endpoint requires a nonempty list with at most ten string entries. A valid batch may contain an invalid URL; the response preserves the position of each item and reports the error for that item while continuing with the others.

A successful report includes the final verdict, risk score, rule checks, structure checks, extracted features, model status, score breakdown, and analysis metadata. The application layer adds timing and history information. An invalid submitted URL is not echoed back by the batch error response, which reduces unnecessary exposure of rejected input.

The front end supports report reopening and export, but those actions do not trigger a visit to the submitted destination. Displayed URLs are text in the application interface, and the report action opens the local assessment.

Table 3.6: Quality requirements and implementation boundaries.

Attribute	Implemented mechanism	Remaining boundary
Explainability	Rule details and score components	Reasons are not per tree ML explanations
Reliability	Clear fallback for unavailable models	Fallback quality is not separately benchmarked
Privacy	Secret redaction and optional history	Unrecognised secrets may remain in paths or values
Robustness	Input limits and structured errors	No exhaustive fuzzing claim
Reproducibility	Pinned source, seeds, hashes, runtime versions	One recorded split does not measure variability
Maintainability	Separate analysis, model, API, and training modules	Feature changes require coordinated updates
Usability	Single and batch modes, visible status, report details	No formal usability study is reported

The analyzer accepts at most 2,048 characters for a submitted URL. The Flask request body limit is 96 KiB. A batch contains at most ten links, and the in memory rate limiter allows up to 60 analysis items per client address within 60 seconds. A batch consumes one unit for each of its entries. History responses are paginated, with a requested limit clamped between one and one hundred records.

These bounds serve different purposes. The URL limit bounds parsing input. The body limit bounds the HTTP payload. The batch limit bounds the work initiated by one submission. The rate limiter bounds repeated local API analysis requests. None of these mechanisms is a substitute for a production deployment design, and the limiter is not shared across multiple server processes.

The strongest privacy related acceptance condition is that runtime analysis does not resolve or fetch the destination. This is checked through source inspection and tests that replace network functions with failures. Secret handling is checked separately because avoiding a network request does not by itself prevent a URL from being written to local history.

A report should remain understandable when the model is unavailable or history cannot be written. For this reason, the application exposes those conditions in the result instead of showing an apparently normal score with missing context. The measured request duration is useful for an individual check, but it is not presented as an aggregate performance benchmark.

Requirements as contracts between stages

Functional requirements describe what the application must do; quality requirements describe how its behavior should remain understandable and dependable. In this project, the two are closely related. Returning a score is functional behavior, while returning enough component information to explain that score supports traceability. Continuing without ML is functional behavior, while clearly marking that condition supports reliability and interpretation.

Acceptance evidence should correspond to a specific contract. A private-history test checks whether a private request creates a database row. A feature-order test checks the schema passed to the model. A batch test checks the relationship between accepted inputs, rejected inputs, and returned indices. These checks are more precise than a general statement that the application works correctly.

The request limits are also contracts. The maximum URL length bounds the accepted input, the body-size limit bounds a request, and the batch-size limit bounds the number of addresses in one operation. The rate limiter counts analysis items, so a batch of ten consumes ten units. These rules affect what a user can submit and should be explained consistently in errors and interface guidance.

Requirements can conflict in practice. Detailed history improves reproducibility but retains more information. A smaller input limit simplifies bounded processing but rejects some long addresses. The project resolves these tensions with optional history and explicit limits. It does not claim that the selected values are optimal for every deployment; they define the current prototype's behavior. [P1, P3]

3.8 SOFTWARE REQUIREMENTS

The supplied model was trained on 26 September 2026. Its metrics file records the data hash, split method, split indices hashes, model parameters, feature order, runtime versions, and training timestamp. The training command writes the model and metrics through temporary files before replacement, reducing the chance of leaving a partially written final file.

Table 3.7: Recorded implementation and verification environment.

Component	Recorded version
Python	3.12.14
scikit-learn	1.8.0
NumPy	2.3.5
SciPy	1.17.0
joblib	1.5.3
Flask used for report verification	3.1.2

The browser interface uses HTML, CSS, and JavaScript served by the same Flask application. SQLite is accessed through the Python standard library. No Node.js build, Docker container, GPU runtime, or paid threat-intelligence API is required for the supplied local demonstration. Installation and verification commands are collected in Appendix A.

Reproducible installation and readiness

The interpreter used to install dependencies should be the interpreter that starts the application. Otherwise, a package can appear installed in one environment while being absent from the running server. The documented commands use the selected Python interpreter to invoke pip, and the project includes a setup check for the ML components. This is an operating requirement for reproducing the supplied model, rather than an additional classification feature.

Readiness has several levels. The web process can start, the health endpoint can respond, the model can load, and the matching evaluation metadata can be available. These states should be inspected separately. The existence of dataset.csv does not itself mean that the trained artifact is valid, and an online service does not establish that model inference is active.

The source project contains the browser assets served by Flask. Opening an HTML file directly changes the origin and request context expected by the application. The intended local workflow is to start the service and open its local address. The report's installation steps follow that workflow so the interface and API are tested together. [P1]

3.9 HARDWARE REQUIREMENTS

The source code, dataset, model artifact, dependency lists, schema, and tests are included. Analysis uses local parsing and model inference. A GPU is not required by the supplied Random Forest implementation. The training script sets two parallel jobs and a fixed random seed. The project does not include a hardware performance study, so specific memory, throughput, or minimum processor guarantees are not claimed.

The implementation separates concerns into small modules. The analyzer handles URL logic, the feature module defines the model vector, the model module handles artifact loading and prediction, and the application module provides HTTP endpoints and persistence. This separation makes maintenance feasible without requiring every change to affect the browser interface.

Distinguishing resource needs from measured performance

The implementation's resource needs can be described without inventing a hardware benchmark. Normal analysis parses a bounded URL, calculates a fixed feature vector, and runs inference with a stored forest. Training works on the full prepared dataset and constructs a new artifact. These are different workloads, so successful interactive use does not establish how long retraining will take on every computer.

The recorded settings use two parallel jobs for the forest. They do not establish a minimum processor model or memory capacity. Similarly, the test-suite duration is the elapsed time of one verification run, not a throughput measurement for a deployed service. A future performance study would need to identify hardware, concurrent requests, cold and warm model states, and the distribution of measured response times.

For a practical demonstration, the important preparation is to confirm that the supplied artifact loads in the intended environment and that the local database directory is writable. These checks directly affect the implemented workflow. They are more defensible than presenting unmeasured hardware requirements as guarantees. [P1, P2, P3]

3.10 PROJECT TIMELINE

The dates below are evidence checkpoints recorded in the supplied artifacts and this report workflow. They are not an invented schedule for undocumented development stages. The project can be reviewed in three stages: design and requirements, implementation and model integration, and final verification with report preparation.

Table 3.8: Documented project and report checkpoints.

Checkpoint	Recorded date	Evidence
Model training	26 September 2026	trained_at in model_metrics.json
Application preview	28 September 2026	Timestamp visible in the supplied report preview
Reproduction and tests	14 October 2026	21-test execution transcript
Template report revision	16 October 2026	This report layout and output review

Review checkpoints and deliverables

The timeline can be understood through the artifacts available at each checkpoint. The training record establishes that a fitted model and associated metrics existed at the recorded timestamp. The interface previews show the application presentation used for the report. The verification transcript records execution of the supplied tests. Report preparation connects these artifacts into an account that can be reviewed.

A design review should examine the intended inputs, component responsibilities, and no-destination-request boundary. An implementation review should trace the feature schema, weighting calculation, status handling, and API contracts. A final evidence review should match every numerical claim to the stored metrics or an explicitly identified calculation. These are review purposes, not invented dates for development work that was not documented.

The expanded report retains those distinctions. Additional explanation improves the account of completed work, but it does not imply that the model was retrained or that another experiment was performed. If a later project stage changes the dataset or artifact, its own checkpoint should record the new identity and evaluation before the report's result tables are revised.

CHAPTER 4

IMPLEMENTATION

4.1 METHODOLOGY

The implementation follows a repeatable sequence: normalize the submitted address, extract numerical features, evaluate direct and structural rules, obtain a local model estimate, combine the components, and create a redacted report. Dataset preparation and training form a separate workflow. A normal analysis request does not retrain the model or retrieve the submitted destination.

An empty path and a slash path represent the same root resource in the feature pipeline. The extractor uses a slash for that case, so https://example.com and https://example.com/ produce equal values for all 24 model inputs. The visible report URL is created separately by the redaction function.

The feature dictionary also contains explanatory fields, such as the estimated registered domain, matched keywords, and analysis time. Only the ordered names in ML_FEATURE_NAMES enter the classifier. Boolean features become numeric zero or one. Finite numerical values are preserved as floats; unsuitable values become zero in the vector conversion helper.

A shared transformation before every prediction

The feature pipeline must produce the same meaning during training and inference. In the training path, a row is read from CSV and passed through normalization and extraction. In the runtime path, the URL comes from an API request and passes through the same operations. The difference is the origin of the input, not the definition of the model columns. This reuse reduces the need to maintain two implementations of the same transformation.

The pipeline also separates representation from presentation. Features are calculated from the accepted URL components, while the returned display URL is redacted. For example, the presence of credentials can be represented as a Boolean warning without returning the credential values. The model receives numerical information about the address, and the user receives an explanation that avoids exposing recognized secrets.

The adapter converts the extracted dictionary into a vector according to ML_FEATURE_NAMES. It does not depend on the insertion order of whatever dictionary a caller supplies. Boolean values become numerical zero or one, and finite

numerical values are retained. The same ordering is recorded in the artifact so the loader can reject a schema mismatch before inference.

An implementation review should follow one input across these boundaries. The accepted string, parsed hostname, feature dictionary, numerical vector, model result, and final report are different representations of one request. Checking how each representation is created is more informative than treating the model call as the whole application. [P1]

4.2 URL VALIDATION AND REDACTION

The normalization function first requires a text value, removes surrounding whitespace, and checks for an empty or excessively long input. It rejects control characters, embedded whitespace, and backslashes. Unsupported schemes such as javascript, data, file, mailto, tel, and ftp are explicitly rejected. Scheme relative input receives an HTTPS prefix, and an address without an explicit scheme is interpreted as HTTPS.

The URL is then split into components. Only HTTP and HTTPS are accepted. A hostname must exist and must satisfy the length and label checks. Internationalized names are converted to an ASCII IDNA representation for consistent hostname handling. IP literals are evaluated with Python's ipaddress module. A numeric dotted string that is not a valid standard IP address is rejected. Port parsing also rejects invalid values.

Python's URL parsing functions separate components but do not by themselves validate an address for an application's intended use [7]. The explicit checks in normalize_url therefore form part of the application's input policy. They should remain covered by tests whenever the parsing logic changes.

The displayed URL hides embedded credentials, replaces recognised sensitive query values, and hides a fragment when present. Sensitive names include token, password, session, secret, api_key, and related forms. The extractor may still use structural information from the original submitted text before this display transformation.

Redaction is intentionally described as recognition based. A secret embedded in a path or under an unfamiliar parameter name may remain visible. Private checks avoid history insertion, but they are not a guarantee that a user supplied secret can never appear in browser memory or a manually exported report.

Validation order and predictable errors

Validation begins with conditions that do not require a complete parsed address: type, emptiness, length, control characters, whitespace, and backslashes. It then handles missing schemes and parses components. Hostname checks, IDNA conversion, IP interpretation, and port validation establish the remaining conditions. This ordering gives malformed input a defined path out of the analyzer before scoring begins.

An address without an explicit scheme is interpreted as HTTPS, while an explicitly unsupported scheme is rejected. These are different cases. The policy is documented so that a user can understand why example.com is accepted but a JavaScript or FTP input is not. The application does not interpret an unsupported scheme as a request to execute another protocol handler.

Redaction then constructs the returned address from accepted components. Credential presence is retained as a warning, recognized sensitive query values are replaced, and a fragment is hidden when present. Non-sensitive query information can remain visible. This allows the report to preserve useful context without simply echoing every original value.

The tests use recognizable dummy secret strings to verify that those strings are absent from the serialized result. That check is more specific than confirming only the visible URL field: a secret could otherwise remain in another response field. However, the implemented recognition rules cannot identify every possible secret embedded in an arbitrary path or unfamiliar parameter. The report retains that limit rather than describing redaction as complete anonymization. [P1, P3]

4.3 LENGTH AND DESTINATION FEATURES

The first twelve entries below follow the exact order in the shared feature schema. Length measurements use characters in the extracted representation, not downloaded content size. The hostname is the ASCII form used by the analyzer. The complete feature order is defined in backend/feature_extractor.py in the accompanying source package [P1].

Table 4.1: Numerical URL features used by the classifier.

No	Feature	Type	Definition
1	url_length	Number	Length after representing an empty root path as a slash
2	hostname_length	Number	Character count of the ASCII hostname
3	path_length	Number	Character count of the path, with slash as the root default
4	query_length	Number	Character count of the query component
5	query_parameter_count	Number	Number of parsed query key and value pairs
6	subdomain_count	Number	Labels before the estimated registered domain
7	https	Boolean	Whether the parsed scheme is HTTPS
8	is_ip_address	Boolean	Whether the hostname parses as an IP literal
9	is_non_public_ip	Boolean	Private, local, reserved, or other nonpublic IP category
10	has_credentials	Boolean	Whether username or password information is embedded
11	has_punycode	Boolean	Whether the ASCII hostname contains the xn-- prefix
12	has_unicode_hostname	Boolean	Whether the original hostname contains non ASCII characters

These values describe the address, not the live service. The https feature means that the URL requests HTTPS; it does not mean that a certificate was inspected or that the destination is trustworthy. Similarly, an IP literal or internationalized name can be legitimate. Their meaning depends on the other observed features and the user's context.

Registered domain estimation uses a small local list of common two label suffixes, including co.in and co.uk, with a last two labels fallback. It is not a complete Public Suffix List implementation. The same approximation affects subdomain counts, brand checks, and the group labels used in dataset splitting. Reported domain disjointness therefore means disjointness under this implemented grouping rule.

The fixed order is part of the model interface. Adding a feature in the middle of the list without retraining would change the meaning of later columns. The loader's feature name check is intended to detect this mismatch before inference.

Worked feature interpretation

For the normalized representation https://example.com/, URL length includes the scheme and the root slash. The hostname is example.com, the path is '/', and the query is empty. These observations describe different parts of the same address. They should not be added together as if they were independent risk scores; they are measurements passed to rules or the model according to the implementation.

The root-path normalization matters because https://example.com and https://example.com/ refer to the same intended root representation in this pipeline. Without a consistent policy, the two spellings could produce different path and URL lengths and potentially different model outputs. The included-model test compares all 24 features for these forms and checks the resulting assessment.

The HTTPS feature records the parsed scheme. It does not mean that the application inspected a certificate or completed a network handshake. Similarly, an IP-address feature reports that the hostname parses as an IP literal. It does not identify the owner of that address or observe its present network behavior. These distinctions keep the feature dictionary aligned with the no-destination-request design.

Subdomain count depends on the estimated registered domain. The helper recognizes a short configured set of two-label suffixes and otherwise uses a simpler fallback. Because this is an approximation, the report uses the same qualification when discussing domain groups in training. An apparently small parsing helper therefore influences both user-facing interpretation and the experimental split. [P1, P3]

4.4 OBFUSCATION AND STRUCTURAL FEATURES

Table 4.2: Numerical URL features used by the classifier.

No	Feature	Type	Definition
13	has_encoded_characters	Boolean	At least one percent encoded byte pattern appears
14	has_double_slash_path	Boolean	Repeated slash characters appear within the path
15	has_unusual_port	Boolean	An explicit port is neither 80 nor 443
16	is_shortener	Boolean	Exact hostname matches a local shortener list
17	suspicious_tld	Boolean	Final domain label occurs in the project's static list
18	dangerous_extension	Boolean	Decoded path ends with a listed executable style extension
19	phishing_keyword_count	Number	Count of distinct listed terms found in decoded lowercase URL text
20	redirect_parameter	Boolean	Query includes a name such as next, redirect, return, or url
21	hostname_entropy	Number	Shannon character entropy of the ASCII hostname, rounded to two decimals
22	digit_ratio	Number	Fraction of URL characters that are digits, rounded to three decimals
23	special_character_count	Number	Count of characters outside the ASCII letters and digits
24	has_sensitive_query_parameters	Boolean	At least one query key matches the secret recognition policy

Hostname entropy summarizes character diversity. A higher value can occur in random looking names, but it can also occur in legitimate names. The implementation

treats entropy as a weak structural signal. Digit ratio and special character count serve a similar role: they describe complexity and are not standalone evidence of a malicious destination.

Keyword detection uses substring matching after decoding and lowercasing. Terms such as login, account, verify, and password are common in both legitimate and deceptive workflows. Their presence can increase a rule score and contribute to the learned pattern model, but the feature definition does not establish the intent of a page.

The shortener, TLD, brand, and file extension lists are project configuration embedded in the source. They are neither live threat feeds nor exhaustive registries. A shortener is not expanded, and a redirect parameter is not followed. The report should explain the observation without implying that the hidden destination has been inspected.

Encoding, text complexity, and contextual meaning

Percent encoding creates two useful views of an address: the written form and the decoded form used for selected checks. The implementation records whether encoded bytes occur and decodes the path when checking configured executable-style extensions. The test involving file%2eexe verifies that the extension check can recognize the decoded period. This is a defined text transformation, not an attempt to download or inspect a file.

Hostname entropy summarizes the distribution of characters in a hostname. The implementation calculates a numerical value from character frequencies and rounds it for the feature representation. A varied string can have a higher value than a repetitive string, but that observation does not establish malicious intent. Legitimate identifiers and generated subdomains can also be complex. The structural engine therefore treats entropy as one configured condition among several.

Digit ratio and special-character count provide another description of form. A path containing many numbers may be an identifier, while a query with many separators may be ordinary application state. The model can learn associations from the historical data, but the feature definition itself remains descriptive. The report's explanations should distinguish a measured property from the conclusion drawn from it.

Keyword matching also requires this distinction. The configured terms are searched within decoded, lowercased URL text. A match records the occurrence of a listed term; it does not read a login page or determine why the term appears. When a warning is displayed, the user can inspect that observation alongside the hostname, other checks, and the final score. [P1]

4.5 DIRECT RULE ANALYSIS

Each triggered rule contributes points to Q. The direct score is capped at 100. A passing HTTPS check contributes no points. Several conditions may apply to the same URL, so the score represents accumulated observations rather than mutually exclusive categories [P1].

Table 4.3: Direct URL rules and assigned points.

Condition	Points	Condition	Points
HTTP scheme	8	IP literal	20
Nonpublic IP literal	25	Embedded credentials	26
Punycode hostname	18	Unicode hostname	12
TLD in static list	12	Known shortener hostname	10
Executable style path ending	35	Percent encoding	8
Repeated path slash	5	Unusual port	8
Listed keywords	5 each, at most 20	Brand term on another domain	14
Redirect style parameter	4	HTTPS scheme	0

Tracing direct points to a displayed explanation

The rule engine builds checks containing identifiers, severity labels, titles, explanatory text, and point values. A reviewer can therefore inspect the direct sum by adding the points of the triggered conditions before applying the cap. The rule explanation is connected to observable URL properties rather than generated from an unexplained final score.

For the middle demonstration address, the direct score is 23. The HTTP scheme contributes eight points, and the three matched terms account, verify, and login contribute fifteen points at five each. The structural engine adds its own contribution later. This example shows why the direct rule score and the heuristic score should be kept separate when explaining an output. [P1, P2]

The rule cap prevents the direct sum from exceeding its intended range, but it also means that additional triggered rules may not increase the capped score. Explanations should still preserve the checks, because they remain relevant observations even when the numerical total is saturated. A risk score is therefore a compressed summary of the conditions, while the check list retains more detail.

The configured points express project policy. They were not estimated by fitting a statistical model to the evaluation set, and their values should not be described as calibrated probabilities. A future change to a point value would change the scoring policy and should be accompanied by arithmetic checks and an evaluation of the

resulting decisions. The current report records the existing values so its examples remain reproducible.

4.6 STRUCTURAL HEURISTICS

Structural conditions produce a separate score S. They are intentionally kept distinct from trained ML inference. The current six conditions total at most 60 points if all are triggered, although the implementation also caps the sum at 100.

Table 4.4: Structural heuristic conditions and points.

Structural condition	Points
URL length greater than 100 characters	12
Query length greater than 120 or more than 8 parameters	10
At least 3 estimated subdomains	12
Hostname entropy greater than 4.0	10
Digit ratio greater than 0.25	8
More than 25 special characters	8

Each check contains an identifier, severity, title, explanatory detail, and point value. The report can therefore show both what was observed and how it affected the heuristic component. These explanations describe the rules. They do not claim to explain the contribution of each feature within the forest for a particular prediction.

The selected points encode project judgement. Their values have not been optimized through a separate validation experiment, and there is no evidence in the supplied project that each individual rule improves the final detector's accuracy.

Structural contribution and possible overlap

The structural engine evaluates six conditions: URL length, query complexity, subdomain count, hostname entropy, digit ratio, and special-character count. Its total is scaled by 0.4 and rounded before being added to the direct rule score. This creates a distinct intermediate value that can be inspected in the response instead of being hidden inside a final formula.

The maximum sum of the six current point values is 60. If all six conditions were triggered, their scaled contribution would be round(0.4 x 60), or 24, before the heuristic cap is applied. This is an arithmetic consequence of the listed rules, not a measured frequency of such URLs. It helps explain why a structural total and a model score use different interpretations even though both are displayed numerically.

Some structural information also appears among the model's inputs. A long address can therefore influence the explicit structural component and the learned estimate. The

components are not statistically independent. The application exposes both so the reviewer can understand how the implemented policy combines them, while leaving the question of optimal weighting to future evaluation.

The direct and structural engines have different explanatory roles. Direct checks identify configured patterns such as credentials or executable-style paths. Structural checks summarize complexity. Neither role makes a statement about downloaded page behavior. A user should read the conditions in context rather than treating a single complexity threshold as a complete decision rule. [P1]

4.7 DATASET PREPARATION

The supplied dataset is a reproducible 40,000 row sample from the DEPHIDES small_dataset/train.txt file. The project records upstream revision e66ef86fc93c6881c194833d479db9862354d7c4 and verifies the source bytes with a SHA 256 checksum before preparation. Labels are retained from the source: legitimate becomes benign, and phishing remains phishing [3, P2].

The preparation script parses the label and URL on each line, rejects invalid inputs, tracks repeated URL values, excludes conflicting labels, selects 20,000 valid unique records from each class using seed 42, and shuffles the selected rows. The CSV includes url, label, source, and source_row, allowing a selected record to be connected to its upstream line.

Table 4.5: Recorded dataset preparation counts.

Preparation item	Recorded value
Source text lines processed	364,200
Source labelled benign entries	201,736
Source labelled phishing entries	162,463
Invalid rows during preparation	1,473
Duplicate rows during preparation	294
Conflicting URL labels removed	0
Selected benign records	20,000
Selected phishing records	20,000
Prepared CSV records	40,000

The line total includes an invalid or unlabeled line, so it is not identical to the sum of the two labelled source counts. Cleaning counts describe the preparation stage. The later training loader found zero invalid or unknown label rows and zero conflicting URLs in the prepared CSV.

These are historical labels, not fresh checks of current website behaviour. The balanced sample supports controlled development, but a real browsing stream will generally have a different class prevalence. Source collection differences may also produce shortcuts that a lexical model can learn. The MIT licence and source attribution are retained in the project documentation and code package.

Dataset acquisition is only part of the explicit preparation script. Analysis of user supplied URLs does not call this script or download new data.

Cleaning stages and retained provenance

The preparation procedure first verifies the bytes of the pinned source file. This check protects the identity of the preparation input: a changed file should not silently produce a sample described as the documented source. After verification, the script reads label and URL text, tracks source counts, validates addresses, and records duplicate or conflicting cases.

Sampling occurs after valid unique records have been grouped by class and conflicts excluded. The configured sample contains 20,000 benign and 20,000 phishing rows. The random seed fixes the selection procedure for the same source and code. The resulting CSV includes source-row information and is accompanied by metadata describing the transformations and output checksum. [P2]

The training loader performs another validation stage when reading the prepared CSV. The preparation counts and training-loader counts should therefore not be mixed. An invalid row removed from the upstream text is different from an invalid row discovered in a later CSV. In the supplied prepared artifact, the training record reports zero invalid or unknown-label rows and zero conflicting URLs.

These stages also clarify what reproducibility means. Using the same labels and sample size is insufficient if the selected URLs differ. A checksum identifies the exact prepared file, while the source metadata explains how it was created. Together they support an audit from published source through transformation to training input. The project does not relabel the source by visiting the historical destinations.

4.8 RANDOM FOREST TRAINING

The training script groups records by the analyzer's estimated registered domain. GroupShuffleSplit selects approximately 20% of these groups for testing with seed 42. The test proportion applies to groups, so the resulting number of URL rows need not equal exactly one fifth of the dataset [6]. Both classes must be present in each partition.

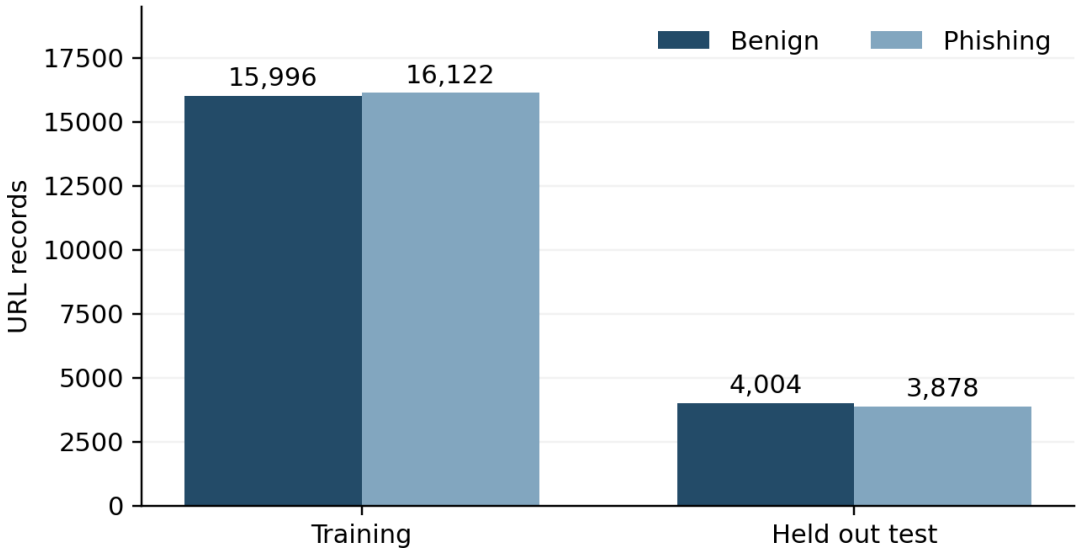


Figure 4.1: Class counts in the supplied training and test partitions

Table 4.6: Class distribution in the training and test partitions.

Partition	Benign	Phishing	Total
Training	15,996	16,122	32,118
Held out test	4,004	3,878	7,882
Complete sample	20,000	20,000	40,000

There are 31,578 estimated registered domain groups and zero overlapping groups between these partitions. This reduces a common route for related addresses to appear in both sets, but it does not remove all similarity, source bias, or errors from the domain approximation.

The Random Forest uses 350 trees, minimum leaf size 2, balanced_subsample class weighting, random state 42, and two parallel jobs. Other estimator parameters use the scikit-learn 1.8.0 defaults. The algorithm fits decision trees and combines their predictions [5]. No hyperparameter search, separate validation set, repeated split study, or cross validation result is recorded for this artifact.

From grouped records to a fitted forest

The training matrix has one row per accepted example and one column per feature in the shared schema. The class vector contains the binary labels, and the group vector contains the analyzer's estimated registered domains. The split operates on groups, which is why the requested 20% test fraction does not imply exactly 8,000 test rows. The actual counts are recorded after the split. [6, P2]

The source checks that both classes occur in both partitions. It can try successive seeds if that condition is not met; the supplied record shows that the requested seed 42 was

also the seed used. The report retains this detail because a fallback seed would identify a different partition from the one a reader might otherwise reconstruct.

For the estimator, the configured 350 trees form one forest. Scikit-learn's classifier combines tree probability estimates to obtain its predicted class probabilities. The balanced_subsample setting derives class weights for each bootstrap sample, and min_samples_leaf controls the minimum leaf size. These are estimator settings; they are not themselves evidence that this configuration is the best one for the task. [5]

The fitted model is evaluated on the held-out rows, and the script records both class predictions and probabilities for the positive label. These outputs support different metrics. The confusion matrix is based on class decisions, while ROC AUC uses the probability ranking. Keeping both outputs avoids trying to reconstruct all evaluation measures from the displayed integer score.

What the stored training record does not include

The supplied artifact contains one documented training and test split. It does not contain a hyperparameter-search history, repeated-split distribution, or independent later-data evaluation. Therefore, the report describes the fitted configuration and its recorded result without presenting a selection process that did not occur. Future comparison work should be recorded as a separate experiment with its own inputs and validation procedure. [P2]

4.9 MODEL LOADING AND FAILURE HANDLING

The model module reads trained_model.pkl from the backend directory. A cache uses the resolved path, file size, and modification time so that an unchanged artifact can be reused and a changed file can trigger reloading. The expected artifact is a dictionary containing a fitted classifier, the exact ordered feature names, and metadata.

The loader checks for a predict_proba method and the class labels zero and one. Prediction finds the index of label one explicitly, so a reversed class order does not silently reverse the meaning of risk. Nonfinite probabilities are rejected, and finite output is limited to the range zero to one. Matching metrics are shown only when their training timestamp and dataset hash agree with the model metadata.

Table 4.7: Model availability and failure handling.

Condition	Reported behaviour
Model file missing or empty	Not trained status; heuristic fallback
Dependency absent or incompatible	Explicit dependency status; heuristic fallback
Artifact or feature schema invalid	Invalid model or feature mismatch status
Prediction raises an error	Prediction failed status; heuristic fallback
History write fails	Analysis returned with a history warning

Artifact format checks support correctness, but the trusted file requirement remains because joblib loading can execute pickle based content [9].

Readiness, cache behavior, and metadata consistency

The loader checks the artifact before making a prediction. An empty file, missing dependency, incompatible library version, invalid object structure, or different feature tuple produces a specific status. This is preferable to substituting an invented probability, because the analyzer can then choose the documented fallback path and the interface can explain why ML was not used.

The cache avoids repeatedly loading an unchanged artifact. Its signature includes the resolved path, file size, and modification time. A changed signature causes another load attempt. This supports local replacement during maintenance, but it is not a cryptographic identity check. The dataset checksum and training metadata serve a different purpose: they connect the model to its evaluation record.

Class ordering is handled explicitly. The inference helper finds label one in classes_ before selecting its probability. The regression test includes reversed class ordering to check this behavior. This prevents a model exposing classes in a different order from reversing the meaning of the returned score. Non-finite probabilities are rejected, while finite values are limited to the expected range before scoring.

The health response shows metrics only when the model metadata and metrics file agree on the training timestamp and dataset hash. A functioning prediction with unavailable matching metrics is different from a failed model load. A user should be able to see that distinction. Loading a trusted artifact and matching its records remain maintenance responsibilities; format checks do not make arbitrary serialized files trustworthy. [P1, P3, 9]

4.10 BROWSER INTERFACE AND HISTORY

The JavaScript client calls the local API with a 20 second request timeout and shows specific messages for connection, timeout, and response failures. It disables submission controls during a check, exposes an analysis busy state, and renders the returned score components. Search requests use sequence identifiers so that an older response does not replace a newer history result.

The interface includes HTML escaping for variable text and guards CSV exports by prefixing values that begin with spreadsheet formula characters. The CSV action in the history view exports the currently displayed page. Model metrics are read from the service response; they are not hardcoded into the interface.

The application includes keyboard interaction for mode tabs, focus handling for reports, and English and Thanglish guide content. These are implemented usability features. A formal accessibility conformance audit and a user study are outside the recorded evidence.

Choose the save setting before submitting a check. A saved assessment can be found through history search and reopened later. A reopened report describes the earlier analysis and is marked as archived. If the model is replaced after the original check, reopening that record does not automatically recalculate its score.

Use a private check when a temporary inspection should not create a database record. The result can still be reviewed and exported. If the backend reports a history warning, the analysis completed but persistence did not succeed; the interface should not be interpreted as evidence that the result is saved.

Enter one address on each line in batch mode, with no more than ten entries. Open individual successful results to inspect their evidence. An invalid item displays an error at its original position. Batch CSV exports reflect the displayed batch. History CSV exports reflect the currently loaded history page, so they should not be described as a complete database backup.

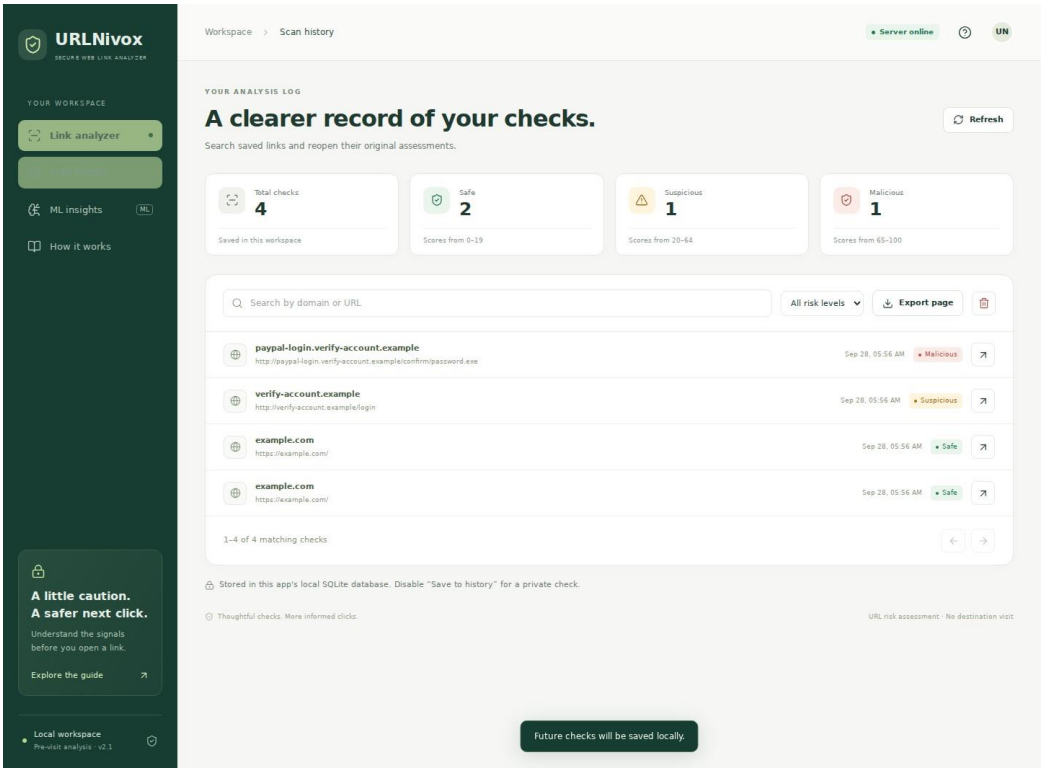


Figure 4.2: Saved assessment history in the supplied project preview. Search, verdict filters, and report reopening support review.

Interface state during analysis and history review

The browser interface manages more than the final display. During a request it disables submission controls, presents an analysis state, and waits for either a result or a meaningful error. The request helper distinguishes connection problems, timeouts, and server responses. These states prevent an old result from being mistaken for a completed new check when the service is unavailable.

History search introduces another ordering issue. A user can change a search term before the previous response arrives. The interface uses request sequence identifiers so an older response does not replace the newer requested view. This behavior supports consistency of the displayed list; it does not change the stored assessments or rerun the model.

The result panel presents the server's component scores, feature values, and status. Variable text is escaped before insertion into generated markup. URLs are shown as text, and report-opening controls open the local assessment rather than the submitted destination. Export actions create copies of the displayed information; they are separate from the optional history write that occurred during analysis. [P1]

An empty history result has two possible meanings: there may be no saved checks, or a search and filter may match none of the existing records. The interface distinguishes these states. The filtered-history screenshot illustrates the latter. This is a small but

useful example of how interface wording can preserve the meaning of the underlying data instead of treating every empty panel as a failure.

4.11 SOURCE CODE

The following excerpts show the core data path without reproducing every file. They are copied from the supplied source, with long lines wrapped only for print. The complete executable files remain in the source package. Imports and supporting functions are defined in those original modules, so the excerpts should be read in their file context.

Feature vector construction

Excerpt from `backend/feature_extractor.py`, original lines 44-62.

```
def _number(value: object) -> float:
    """Convert a scalar feature to a safe finite float."""
    if isinstance(value, bool):
        return float(value)
    if isinstance(value, (int, float)):
        number = float(value)
        return number if isfinite(number) else 0.0
    return 0.0


def to_ml_feature_vector(features: Mapping[str, object]) -> list[float]:
    """Return features in the exact order stored with the trained model."""
    return [_number(features.get(name, 0.0)) for name in ML_FEATURE_NAMES]


def to_ml_feature_row(features: Mapping[str, object]) -> dict[str, float]:
    """Return a named numeric row, useful when inspecting experiments."""
    vector = to_ml_feature_vector(features)
    return dict(zip(ML_FEATURE_NAMES, vector, strict=True))
```

Model inference

Excerpt from `backend/ml_model.py`, original lines 128-161.

```
def predict_url_risk(features: dict[str, object]) -> dict[str, Any]:
    """Predict malicious/phishing URL-pattern probability from local
    features."""
    artifact, unavailable = _load_artifact()
    if unavailable is not None:
        return unavailable

    assert artifact is not None
    try:
        probability = _positive_probability(artifact["model"],
    to_ml_feature_vector(features))
    except Exception:
        return _unavailable("prediction_failed", "The trained ML model could
    not score this URL.")

    score = round(probability * 100)
    if score >= 70:
        prediction = "Malicious pattern"
```

```
    elif score >= 35:
        prediction = "Suspicious pattern"
    else:
        prediction = "Benign pattern"

    metadata = artifact.get("metadata") if
    isinstance(artifact.get("metadata"), dict) else {}
    return {
        "available": True,
        "status": "available",
        "message": "Random Forest URL-pattern model prediction.",
        "risk_score": score,
        "prediction": prediction,
        "confidence": score if probability >= 0.5 else 100 - score,
        "model": metadata.get("model_name", "Random Forest"),
        "trained_at": metadata.get("trained_at"),
        "dataset_rows": metadata.get("dataset_rows"),
        "dataset_source": metadata.get("dataset_source"),
        "model_probability": probability,
    }
```

Final verdict and hybrid assessment

Excerpt from `backend/analyzer.py`, original lines 356-387.

```
def _verdict(score: int) -> tuple[str, str]:
    """Map a numeric score to the user-facing overall risk assessment.

    A URL is called malicious only when several strong indicators are present.
    The labels describe URL-text signals observed by this offline analyser;
    they
    are not proof of malicious website content.
    """
    if score >= 65:
        return "Malicious", "malicious"
    if score >= 20:
        return "Suspicious", "suspicious"
    return "Safe", "safe"


def analyze_url(raw_url: str) -> dict:
    """Analyse a URL without sending any network request to it."""
    value, parsed, hostname, ascii_hostname, port = normalize_url(raw_url)
    features = extract_features(value, parsed, hostname, ascii_hostname, port)
    rules = rule_based_analysis(features)
    patterns = pattern_analysis(features)
    # Direct risk indicators carry more weight than cosmetic URL structure.
    # Keeping the calculation explicit makes the score easy to explain in the
    UI.
    risk_score = min(rules["rule_score"] + round(patterns["signal_score"] *
    0.4), 100)
    heuristic_score = risk_score
    ml = predict_url_risk(features)
    risk_score = round(heuristic_score * 0.60 + ml["risk_score"] * 0.40) if
    ml["available"] else heuristic_score
    verdict, risk_level = _verdict(risk_score)
    display_url = redact_url(parsed, ascii_hostname, port)

    all_reasons = rules["reasons"] + [
        reason for reason in patterns["reasons"] if reason not in
    rules["reasons"]
```

Single URL API endpoint

Excerpt from `backend/app.py`, original lines 142-152.

```
@app.post("/api/analyze")
def analyze():
    try:
        data = _payload()
        if not isinstance(data.get("url"), str):
            raise ValueError("The 'url' field must be text.")
        if _is_rate_limited():
            return _json_error("Limit reached: 60 links per minute. Try again
    shortly.", 429)
        return jsonify(_run(data["url"], save=data.get("save_history", True)))
    except ValueError as error:
        return _json_error(str(error))
```

Estimator configuration and evaluation

Excerpt from `backend/train_model.py`, original lines 128-149.

```
    model = RandomForestClassifier(
        n_estimators=350,
        min_samples_leaf=2,
        class_weight="balanced_subsample",
        random_state=random_state,
        n_jobs=2,
    )
    print(f"Training on {len(train_indices):,} URLs; evaluating on
    {len(test_indices):,} held-out URLs...", flush=True)
    model.fit(matrix[train_indices], labels[train_indices])
    predicted = model.predict(matrix[test_indices])
    probabilities = model.predict_proba(matrix[test_indices])[:,
    list(model.classes_).index(1)]
    true_labels = labels[test_indices]
    tn, fp, fn, tp = (int(value) for value in confusion_matrix(true_labels,
    predicted, labels=[0, 1]).ravel())

    performance = {
        "accuracy": round(float(accuracy_score(true_labels, predicted)), 6),
        "precision": round(float(precision_score(true_labels, predicted,
    zero_division=0)), 6),
        "recall": round(float(recall_score(true_labels, predicted,
    zero_division=0)), 6),
        "f1_score": round(float(f1_score(true_labels, predicted,
    zero_division=0)), 6),
        "roc_auc": round(float(roc_auc_score(true_labels, probabilities)), 6),
        "false_negative_rate": round(float(fn / (fn + tp)) if fn + tp else
    0.0, 6),
        "confusion_matrix": {"true_benign": {"predicted_benign": tn,
    "predicted_risky": fp}, "true_risky": {"predicted_benign": fn,
    "predicted_risky": tp}},
```

These excerpts keep the distinction between the model prediction and the overall verdict visible. The classifier returns a probability-derived score, the analyzer calculates the hybrid score, and the API handles validation and persistence around that result. Reproducibility and the complete repository layout are documented in Appendix A.

Reading the selected code as one execution path

The feature-vector excerpt establishes the input order. The inference excerpt shows how the classifier supplies a probability-derived score and an availability state. The analyzer excerpt then shows the weighting formula and verdict thresholds. Finally, the API excerpt shows how a request enters that calculation. Reading these snippets in this order connects the interface result to the functions responsible for it.

The snippets are deliberately short enough to support explanation during project review. Supporting imports, validation helpers, configured lists, and storage functions remain in the complete source package. The source locations identify where a reviewer can inspect those surrounding definitions. The printed excerpts should not be treated as independent programs to run without their modules.

Code review and execution evidence serve different purposes. A formula visible in source explains intended behavior; a test verifies its result for specified inputs. The report includes both because neither substitutes for the other. The test suite checks the integration of the model contribution and reproduces the stored metrics using the actual artifact. [P1, P3]

4.12 INPUT

The following request is for the local API. The example domain is illustrative and is not contacted by the analyzer. Setting save_history to false allows a demonstration without creating a history record.

```
{
  "url": "https://example.com",
  "save_history": false
}
```

The response includes url, hostname, verdict, risk_score, rule_based, signal_based, features, ml, score_breakdown, analysis, and history_id. The model object identifies availability separately from its score. The score breakdown carries the active weights and final thresholds, allowing a client to explain how the assessment was assembled.

Table 4.8: Runtime input contract for individual and batch analysis.

Input item	Accepted form	Validation
Single URL	JSON string in the url field	HTTP or HTTPS; at most 2,048 characters
Save setting	Boolean save_history	True saves; false returns a private assessment
Batch	List of one to ten URL strings	Each item receives its own result or error
Request body	application/json	At most 96 KiB

Single and batch request semantics

For an individual analysis, the URL field must contain text and the optional save_history field must contain a Boolean. Omitting the preference uses the application's default behavior. A string such as 'false' is not the same type as the Boolean false and is rejected by the payload contract. This avoids ambiguous interpretation of a setting that controls whether a record is retained.

Batch input has an additional outer contract: it must be a list containing one to ten strings. The API validates that shape before processing the entries. Individual URL errors are then reported at their original indices, while valid entries receive reports. The succeeded field counts completed assessments rather than assuming that every member of an accepted batch was valid.

The rate limiter counts items before running the batch, so one request containing ten addresses is not treated like one single-address operation. This keeps the stated limit connected to the amount of analysis requested. The current implementation is local and process based; a deployment with several workers would require a deliberate decision about shared rate accounting.

Malformed input should be tested as input, not assigned a risk score merely to produce a result. The API's structured errors distinguish an unacceptable request from a valid assessment with a high or low score. That distinction is part of the meaning of the output contract. [P1, P3]

4.13 OUTPUT

Start with the overall score and verdict, then inspect the reasons rather than treating the label as a complete answer. The rule section explains visible URL properties. The structure section explains complexity related signals. The ML section shows whether the trained artifact was used and supplies its own pattern estimate. The feature table allows a technical reviewer to inspect the numerical inputs.

Table 4.9: Verified example outputs from the supplied application.

Demonstration URL	H	M	Final	Verdict
https://example.com/	0	3	1	Safe
http://verify-account.example/login	27	94	54	Suspicious
http://paypal-login.verify-account.example/confirm/password.exe	81	94	86	Malicious

These values were obtained from the included model during report verification. H is the heuristic score, and M is the rounded model score. The addresses are illustrative inputs, and their destinations were not visited. They demonstrate the three implemented final bands; they are not additional labelled benchmark records.

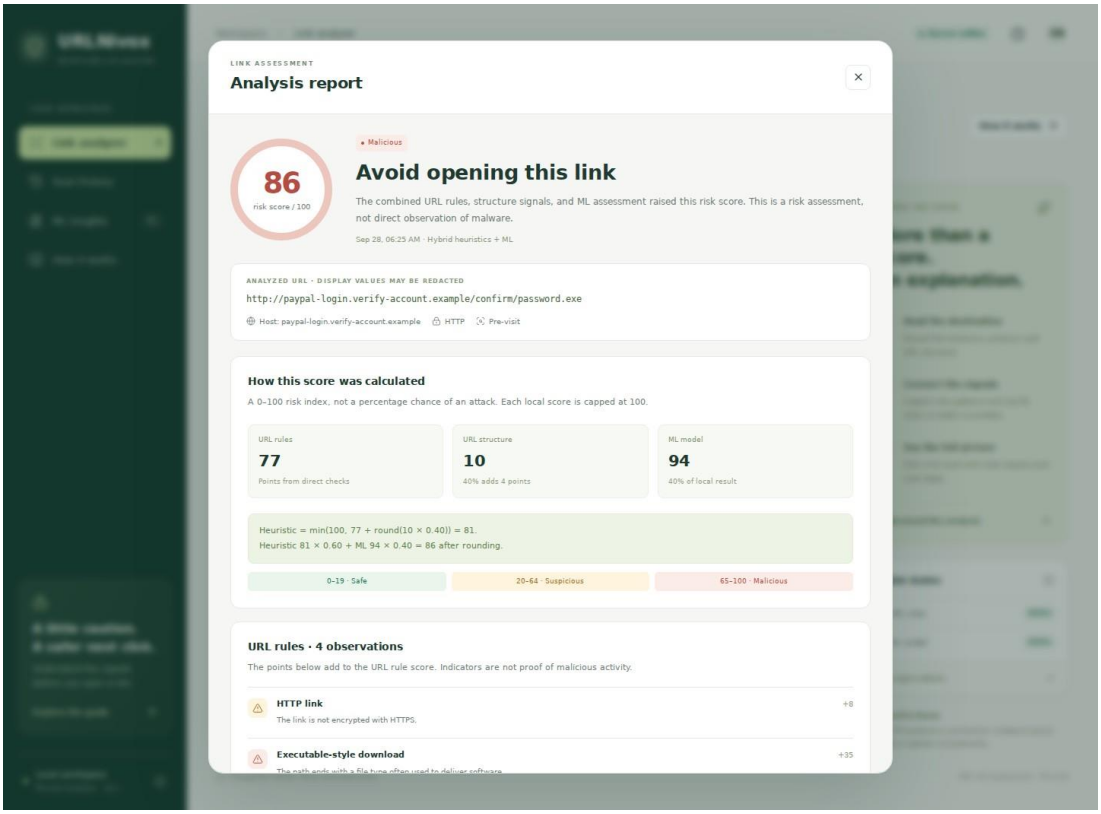


Figure 4.3: Analysis output for the executable-style example. The preview displays a final score of 86, direct rule score 77, structure score 10, and ML score 94.

The preview was supplied with the project. Its displayed calculation agrees with the independently reproduced example in this report. The example domain is used to demonstrate the implementation; it is not a claim about an active website. The detailed output contains the final verdict, extracted features, rule reasons, model status, score breakdown, and history status.

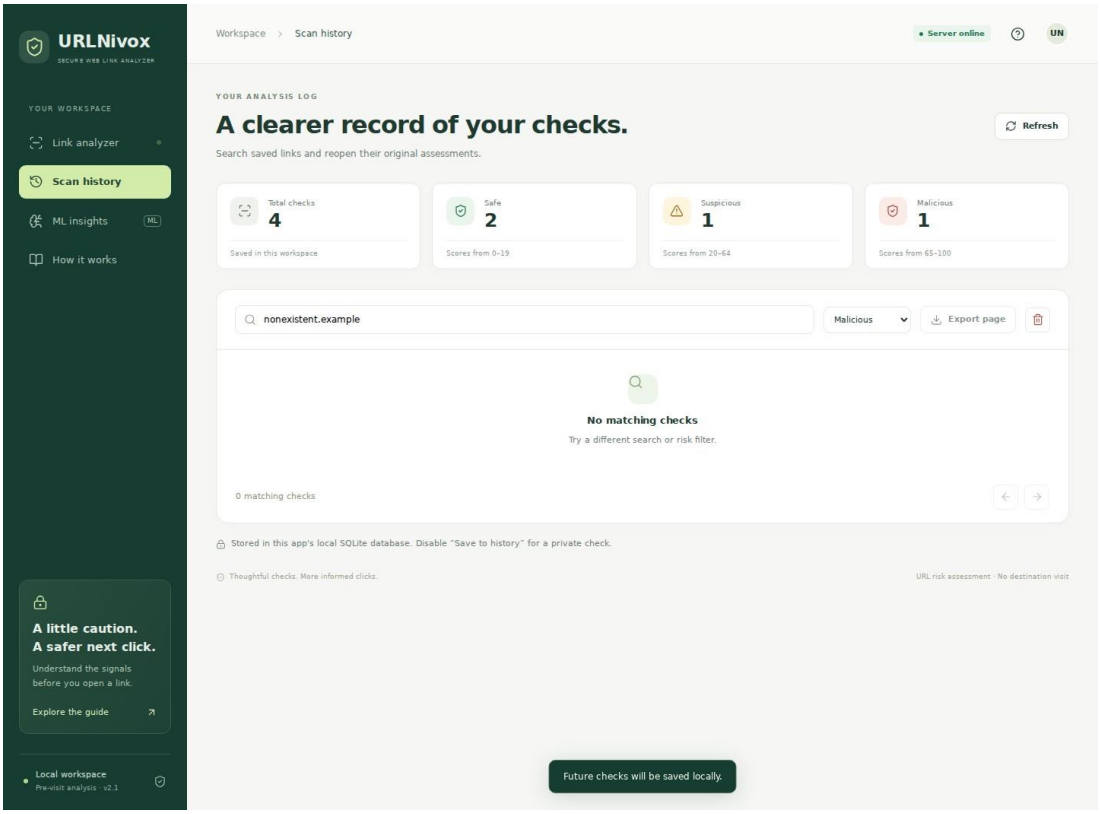


Figure 4.4: Filtered history from the supplied project preview. A search with no matching saved assessment shows an empty result state.

Interpreting the three demonstration results

The first example has a heuristic score of zero and a model score of three. The final score of one shows that a low overall result can still include a small nonzero model estimate. The absence of major rule warnings is not represented as proof that the destination was inspected and cleared. It means that the implemented conditions produced little warning evidence for that URL text.

The middle example is particularly useful for explaining the hybrid calculation. Its model component is 94, but its heuristic component is 27, producing a final score of 54. The overall Suspicious verdict therefore differs from the model's Malicious pattern description. Both outputs are consistent with their own thresholds and roles. Treating either label as a duplicate of the other would hide the policy implemented by the combination.

The executable-style example has strong heuristic evidence as well as a high model estimate. Its final score is 86. The detailed report shows the direct rule score, structure score, and model score separately so the result can be reconstructed. The screenshot and the recorded execution example agree on this arithmetic. [P1, P2]

These examples demonstrate application behavior, not statistical coverage. Three persuasive outputs cannot establish accuracy, precision, or recall. Those measures

come from the labelled held-out partition in Chapter 5. The examples instead answer a different review question: whether the displayed result follows the implemented calculation and explains its inputs.

4.14 CALCULATION

Let Q be the capped direct rule score and S the structural score. The heuristic score is H = min(Q + round(0.4 × S), 100). Let p be the model probability for label 1, and let M = round(100 × p). When the model is available, the final score is R = round(0.6 × H + 0.4 × M). Otherwise, R equals H. The implementation uses Python's round function at these stages [P1].

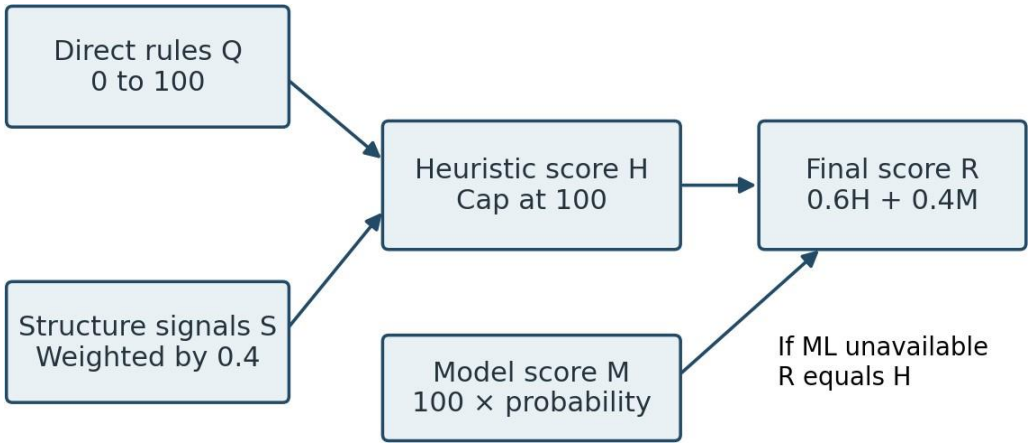


Figure 4.5: Components of the hybrid score with rounding applied in code

Table 4.10: Overall risk score bands in the application.

Final score R	Overall verdict	Meaning within this application
0 to 19	Safe	Few warning patterns in the URL text
20 to 64	Suspicious	Additional review is appropriate
65 to 100	Malicious	Several strong URL indicators may be present

The final bands above are not the ML model's descriptive pattern bands. The model component uses 35 and 70 to describe benign, suspicious, or malicious patterns. The binary classifier metrics use the classifier decision associated with threshold 0.5. These three uses must not be mixed when explaining results.

For http://verify-account.example/login, the verified heuristic score is 27 and the model score is 94. The final rounded score is 54, giving the overall verdict Suspicious. The model's own pattern label is Malicious pattern. This is an expected consequence of the separate scales, rather than a contradictory implementation result.

Worked example

For the executable-style example, the direct rule sum is Q = 77 and the structural score is S = 10. The structure contribution is round(0.4 x 10) = 4. Therefore H = min(77 + 4, 100) = 81. The model score is M = 94. The final result is R = round(0.60 x 81 + 0.40 x 94) = round(86.2) = 86. Because R is at least 65, the overall verdict is Malicious.

For the simple HTTPS example, H = 0 and M = 3, so R = round(1.2) = 1 and the overall verdict is Safe. For the middle example, H = 27 and M = 94, so R = round(53.8) = 54 and the overall verdict is Suspicious. The same model score can therefore appear with different overall verdicts when the heuristic evidence differs.

The effect of weighting and rounding

The formula is applied in stages. First, the structural contribution is rounded and added to the direct score to obtain the capped heuristic score. Second, the model probability is converted to its integer score. Third, the weighted heuristic and model scores are combined and rounded. Reproducing only the final weighted expression from an unrounded probability can therefore differ from reproducing the actual implementation.

When ML is unavailable, the final result equals the heuristic score. This is a change in the active evidence, not a model prediction of zero. Treating an unavailable model as zero while retaining the 0.4 weight would reduce the result without justification. The explicit fallback avoids that particular error and reports the active engine in the analysis metadata.

The formula also permits simple hypothetical checks. If H is zero and M is 100, the weighted result is 40, which lies in the overall Suspicious band. If H is 100 and M is zero, the result is 60, also Suspicious. These are arithmetic examples of the combination rule, not observations from the test dataset. They show why a strong component score does not automatically determine the overall label.

Thresholds should be read after the final rounding operation. A boundary decision depends on the stored formula, its intermediate values, and the final bands together. For maintenance, a change to weights, rounding order, or thresholds should be treated as a scoring-policy change and checked against both arithmetic examples and a stated evaluation procedure. [P1]

CHAPTER 5

RESULTS AND DISCUSSION

5.1 VERIFICATION AND TESTING

The supplied automated suite was run for this report using Python 3.12.14, Flask 3.1.2, and the recorded ML dependency versions. All 21 tests passed. The suite completed in 4.302 seconds in the report preparation environment; this is test execution time, not a user request latency benchmark [P3].

Table 5.1: Executed automated tests and representative coverage.

Test group	Tests	Representative coverage	Result
AnalysisTests	8	Invalid URLs, normalization, fallback arithmetic, redaction, encoded files, model class order, mocked network access	Passed
ApiTests	8	Root and health, payloads, batches, history, schema migration, deletion, limits, network restrictions	Passed
IncludedModelTests	5	Model readiness, provenance, real inference, root equivalence, held out metric reproduction	Passed
Total	21	Supplied automated suite	Passed

The invalid input tests exercise empty values, unsupported schemes, spaces, backslashes, malformed domain labels, invalid ports, control characters, oversized strings, and invalid numeric IP text. Normalization tests include Unicode hostnames and IPv6. The secret test checks that recognized credential and query secrets do not appear in the serialized report.

The API tests use a temporary database and verify both saved and private checks. They also cover batch partial success, an excessive batch size, incorrect payload types, the body size limit, literal history search, pagination, and an earlier schema without report_json. Rate limiting and deletion are checked independently of the original user history database.

The model tests exercise the actual artifact, verify matching metadata, and reproduce the evaluation. Network functions are replaced with exceptions during relevant checks. A passing test demonstrates that the tested path did not invoke those functions; it is not an exhaustive proof about every future code change.

The recorded run is a backend and integration verification. It is not a load test, a formal penetration test, a complete browser compatibility study, or an accessibility

certification. The front end's source behaviour is documented separately. The test transcript is included in the accompanying delivery notes for auditability.

Reading the verification evidence

The three test groups answer different questions. Analysis tests check the calculations and transformations applied to a URL. API tests check whether callers can reach those operations through the documented request contract and whether storage behaves correctly. Included-model tests connect the application to the supplied trained artifact and recorded dataset. A passing result in one group therefore complements the others; it does not replace them.

The temporary database used by the API tests is useful because each check can establish its own starting state. A history test can insert a known record, search for it, and remove it without depending on records from an earlier demonstration. Similarly, a private-analysis test can compare storage before and after the request. These checks turn requirements about persistence into observable behaviour.

Failure-path tests are especially relevant to an optional model. The expected outcome is not always a successful ML prediction. A missing or incompatible model should lead to a clearly identified heuristic result, while a failed history write should leave the completed assessment available. Correct handling means returning the appropriate status and preserving the calculations that remain valid.

The recorded transcript is dated 14 October 2026. The expanded explanations in this report do not represent an additional test run. The value of that transcript is its connection to the documented source, environment, and artifact; any later code or dependency change should receive its own verification record. [P3]

5.2 EVALUATION PROTOCOL

The reported experiment evaluates the binary Random Forest classifier on 7,882 held out historical URL records. The positive class is phishing or risky, encoded as one. Predictions come from the supplied model artifact; probabilities for label one are used for ROC AUC. The current tests reproduced the stored performance values to six decimal places on 14 October 2026 [P2, P3].

Table 5.2: Binary classifier performance on 7,882 held-out records.

Metric	Result	Interpretation
Accuracy	89.96%	Correct predictions among all test records
Precision	88.29%	Actual phishing among records predicted phishing
Recall	91.77%	Detected phishing among actual phishing records
F1 score	0.9000	Harmonic mean of precision and recall
ROC AUC	0.9677	Ranking separation across decision thresholds
False negative rate	8.23%	Actual phishing predicted benign
False positive rate	11.79%	Actual benign predicted phishing

The first six metrics are retained in model_metrics.json. False positive rate is calculated here from 472 false positives divided by 4,004 actual benign records. Percentages are rounded for presentation; the machine readable report retains six decimal places for its stored rates.

Using TP, TN, FP, and FN for true positives, true negatives, false positives, and false negatives: accuracy is (TP + TN) divided by all records; precision is TP divided by (TP + FP); recall is TP divided by (TP + FN); and F1 is twice precision times recall divided by their sum. The false negative rate is FN divided by (FN + TP).

The classifier recognizes useful patterns in this selected dataset. It also makes 791 errors, so the result does not justify automatic trust in every individual classification. No confidence interval or repeated experiment distribution is reported. The 89.96% value must not be presented as the measured accuracy of the full three band hybrid system or as a rate for current live browsing.

From experiment definition to reported numbers

The unit counted in the metric table is a URL record. The unit used to separate the partitions is an estimated registered-domain group. Keeping these units distinct explains both the test size and the scope of the claim. There are 7,882 test records, but they are not 7,882 separately sampled websites with freshly inspected behaviour. They are historical labelled entries assigned through the recorded grouping procedure.

Each metric answers a particular question. Accuracy gives the fraction of all decisions that agree with the labels. Precision asks how many phishing predictions are correct. Recall asks how many labelled phishing records are recovered. F1 combines precision and recall into one value but does not include true negatives directly. The confusion

matrix is therefore needed alongside the metric summary to show the actual distribution of outcomes.

ROC AUC uses the ordering of model probabilities across thresholds. It is not the percentage of correct decisions at the application bands. The recorded AUC of 0.9677 and accuracy of 89.96% can coexist because they summarize different aspects of the same experiment. Selecting a threshold for a particular use case would require a separate policy decision supported by validation data.

Why prevalence affects interpretation

The test subset contains 3,878 phishing records out of 7,882, or about 49.20%. This composition is close to the balanced source sample. A browsing stream with many more benign addresses would change the number of false alarms relative to correct warnings, even if the class-specific error rates happened to remain unchanged. That assumption is not established for this project.

As an arithmetic illustration only, suppose 10,000 records contained 1% phishing and the measured recall and false-positive rate transferred unchanged. Approximately 92 phishing records would be detected, while about 1,167 benign records would receive a phishing prediction. The resulting precision would be about 7.3%. This is not a deployment forecast or an additional result; it demonstrates why the measured 88.29% precision cannot be carried into a population with a different class balance without further evidence.

5.3 CONFUSION MATRIX AND ERROR ANALYSIS

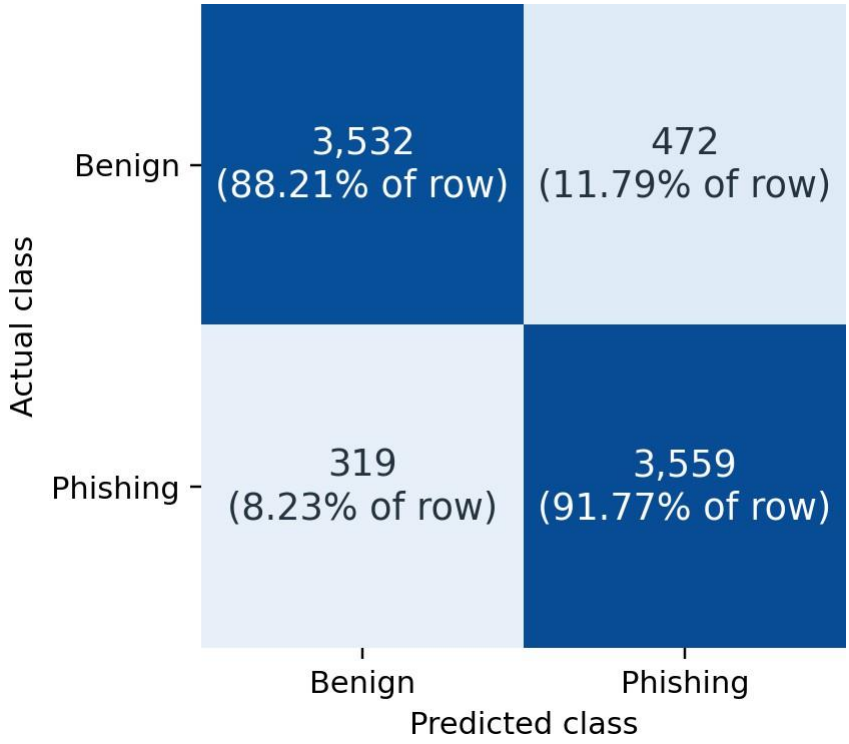


Figure 5.1: Binary classifier confusion matrix on 7882 held out URLs

The model correctly classified 3,532 benign records and 3,559 phishing records. It classified 472 benign records as phishing and missed 319 phishing records by predicting benign. These counts sum to the test set size and produce 7,091 correct decisions [P2]. The percentages in the figure are normalized within each actual class.

A false negative is a risky labelled address that the model treats as benign. In an inspection tool, this is a reason to avoid presenting the output as a guarantee. A false positive is a benign labelled address that receives a risky prediction. Frequent false positives can make a tool less useful if users stop paying attention to warnings.

These two errors have different consequences, but their relative cost depends on the deployment setting. The project does not specify a validated cost model. Changing the binary threshold or the final hybrid bands could change the tradeoff, and such a change should be evaluated on a reserved validation set before final testing.

The confusion matrix provides aggregate counts. It does not identify the cause of each error or prove that a particular rule would correct it. A future error review should group mistakes by source, collection time, domain pattern, and relevant feature combinations. Such review should use the URL text as data and should not require opening a potentially harmful destination.

Checking the confusion matrix by hand

The actual benign row contains 3,532 true negatives and 472 false positives, giving 4,004 records. The actual phishing row contains 319 false negatives and 3,559 true positives, giving 3,878 records. The predicted phishing column contains 4,031 records, and the predicted benign column contains 3,851. Both row and column totals sum to 7,882, providing a simple consistency check on the presentation.

The accuracy numerator is 3,532 plus 3,559, which gives 7,091 correct decisions. Dividing by 7,882 produces approximately 89.96%. Precision divides 3,559 by the 4,031 phishing predictions, while recall divides the same 3,559 by the 3,878 actual phishing records. These different denominators explain why precision and recall are close but unequal.

There are 472 false positives and 319 false negatives. Their sum is 791, or approximately 10.04% of the test subset. About 59.67% of the errors are false positives, but this proportion does not mean that false positives are necessarily more costly. Error frequency and error consequence are separate questions. The application has no recorded experiment assigning costs to missed phishing or unnecessary warnings.

A practical error-review worksheet

A future review can retain the record identifier, historical label, predicted class, positive-class probability, estimated domain group, and extracted features for each mistake. The reviewer can then assign descriptive categories such as a short credential-themed path, a complex benign query, or a nonstandard host representation. Categories should be assigned from the available text and documented criteria, without claiming that a destination was inspected.

The review should include correctly classified records with similar features. Looking only at mistakes can suggest a rule that fixes a few selected cases while creating new errors elsewhere. Candidate changes should be selected using training or validation material, with the final test partition reserved for the stated evaluation. This proposed review was not performed to produce the reported matrix.

5.4 FEATURE IMPORTANCE

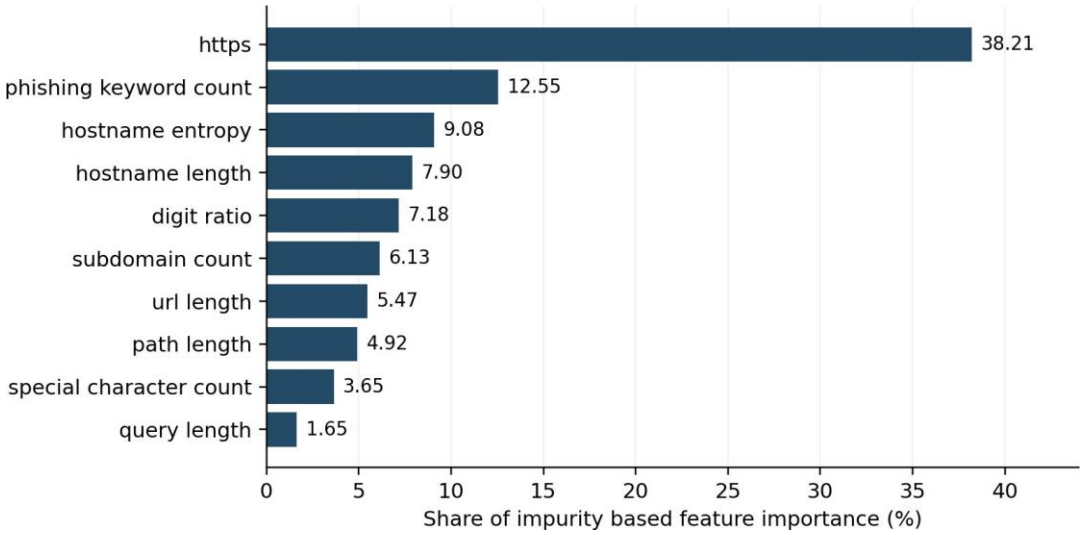


Figure 5.2: Ten largest impurity based feature importances from the trained forest

The largest recorded importance is https at approximately 38.21%, followed by phishing_keyword_count at 12.55%, hostname_entropy at 9.08%, and hostname_length at 7.90%. These values come from the trained forest's feature_importances_ attribute [P2]. The plot shows the ten largest entries, not all 24 features.

The model relies heavily on the scheme related feature in this experiment. This is an observation about the artifact, not evidence that HTTPS itself establishes legitimacy. It suggests an important follow up experiment: compare performance with and without the https feature on independent and later data. That would test whether the model is learning a collection shortcut that may not persist in deployment.

Impurity based importances are global model summaries. They are not causal effects, calibrated risk contributions, or explanations of an individual URL. Correlated features and features with many possible values can complicate their interpretation. Held out permutation importance is a useful complementary analysis, but it has not been run as part of the supplied evaluation [10].

Several features have zero or very small recorded importance. This does not show that the corresponding property is universally irrelevant. It means that the feature contributed little or no measured split importance in this particular fitted forest. The rule component can still flag such properties explicitly.

Connecting importance to a follow-up experiment

The prominent HTTPS importance motivates a specific comparison: train the same estimator on the same training partition once with all features and once with the HTTPS feature excluded. Keep the evaluation partition and other settings fixed, then compare both the summary metrics and the individual disagreements. This would isolate the effect of removing that input more clearly than comparing unrelated models trained on different samples.

A second comparison on independently collected data would address transfer beyond the source dataset. If the feature remains useful there, it may represent a stable pattern in the chosen data. If performance changes sharply, the result would support closer examination of collection differences. Neither outcome can be inferred from the current importance chart alone.

For an individual report, the rule explanations remain easier to trace: the reader can connect an observed URL property to a documented point value. The forest importance chart describes aggregate training behaviour. Presenting both is useful when their purposes are kept clear, but a high global importance should not be displayed as the reason for every prediction.

5.5 REPRODUCIBILITY OF THE RESULTS

The prepared dataset SHA 256 is shown below. This value identifies the exact CSV used for the stored evaluation. A file with the same name but different bytes is a different dataset and should receive a new training record.

```
131 9c70f64f817bc 93f9d0e3433ce60751bba
10e281c2a727 995a 91ba4647b 9a
```

The two printed lines form one continuous 64 character hash. The full machine readable value is in dataset_metadata.json and model_metrics.json. The source file checksum and training and test index checksums are also retained there.

For metric verification, use the included dataset and model first. The test suite rebuilds the feature matrix, reconstructs the grouped split, obtains predictions from the existing artifact, and compares the recomputed metrics with the stored report to six decimal places. This verifies consistency without changing the model being documented.

For a new training run, use a separate output path and retain the original artifact until the new results have been reviewed. A new run receives a new timestamp and may create different artifact bytes even with fixed seeds. Dependency versions, input data, feature code, grouping logic, and estimator parameters all belong to the experiment record. Loading across different scikit-learn versions is not a supported compatibility guarantee [9].

Separating reproduction from retraining

Reproduction checks the result of the supplied artifact. It reconstructs the recorded feature matrix and partition, predicts with that artifact, and compares the calculated values with the stored metrics. Retraining produces a new artifact. Even when both operations use the same input data and seed, they answer different questions and should retain separate records.

When a reproduced value disagrees with the report, the first checks are the dataset hash, feature order, grouping logic, dependency versions, and identity of the loaded model. A file named trained_model.pkl is not sufficient evidence that the expected artifact was loaded. The metadata and readiness response help connect a filename to its recorded experiment.

Rounding also matters when comparing printed values. The PDF gives readable percentages and four decimal places for selected metrics, while the machine-readable values provide greater precision. A comparison should use the stored values and the documented tolerance before rounding for display. Repeatedly rounding intermediate calculations can create a small presentation difference even when the underlying counts agree.

5.6 DEMONSTRATION AND TROUBLESHOOTING

1. Start the local service and check that the model insights page reports a loaded model, 40,000 labelled URLs, and the recorded split.
2. Disable saving if the demonstration should leave no history. Analyze https://example.com and inspect the low score and feature table.
3. Analyze http://verify-account.example/login and explain the separate heuristic, model, and final scores.
4. Analyze the executable style example in Section 4.13 and identify the direct rule evidence behind its higher score.
5. Submit a small batch containing one valid address and one unsupported scheme. Show that the valid item succeeds and the invalid item receives its own error.
6. Enable saving for a non sensitive example, reopen it from history, and explain the archived flag. Export the report JSON if required.

```
python -m unittest discover -s tests -v
```

The supplied suite uses temporary history storage for its API checks. Run it from the project root with the same interpreter that has the required dependencies installed. The expected result for the delivered version is 21 passing tests.

Table 5.3: Troubleshooting checks for a local demonstration.

Symptom	Check or corrective action
Browser cannot reach the analyzer	Confirm app.py is running and open the correct local port
Port is already occupied	Set LINKGUARD_PORT to an available value and use that URL
ML dependencies are missing	Install requirements.txt using the running interpreter
Model version mismatch	Restore the pinned ML dependency versions
Model missing or invalid	Restore the trusted supplied artifact or retrain deliberately
History cannot be saved	Check that the database directory is writable
Batch rejected	Supply one to ten string URLs and valid JSON
Rate limit reached	Allow the 60 second window to clear before retrying

Troubleshooting should preserve the distinction between an analysis failure and a storage or model availability warning. A fallback assessment can still be returned when the model is unavailable, and a completed analysis can still be returned when history persistence fails.

Explaining a demonstration as it runs

A useful demonstration follows one complete assessment before switching modes. Enter an example, identify whether ML is available, read the overall verdict, and then connect the visible reasons to the component scores. The intermediate example is particularly useful because its high model score and moderate final score show how the weighting actually affects the result.

History can then be demonstrated using an example that contains no private information. Save it, open the archived record, and explain that the stored report describes the earlier check. A separate private request should leave no new history entry. This sequence makes the save setting observable instead of treating it as a label in the interface.

If a demonstration fails, diagnosis should follow the failed stage. A browser connection error calls for checking the running service and local port. An unavailable-model status calls for checking dependencies and the artifact. A history warning calls for checking the database path and write access. Reinstalling every component before identifying the stage can hide the original cause and make the demonstration harder to reproduce.

5.7 PRIVACY AND ROBUSTNESS

The no destination request property reduces one form of external disclosure. It does not make every submitted URL non sensitive. Private mode stops the application's history write for that check, while manual exports create their own files. Redaction covers named patterns; it does not inspect all possible application specific secret formats.

The delivered code package starts without the original history database. The schema and application recreate an empty database when history is first used. This preserves the software's normal operation while avoiding redistribution of earlier saved checks.

The implemented protection mechanisms include structured validation, bounded request sizes, explicit error responses, secret redaction, and parameterized database operations. They were reviewed through the supplied source and automated tests. Passing those checks establishes the tested behaviour; it does not convert the local prototype into a multiuser security service.

Data handling across the workflow

The submitted text first exists in the browser and request body. The analyzer extracts features and prepares a redacted display value. When saving is enabled, the report is retained in the local history database. A user export creates an additional file under the user's control. These are distinct locations with distinct lifetimes, so the absence of a destination request does not imply that no local copy exists.

The save setting applies to application history for that request. It does not erase the browser input field, revoke a previously downloaded export, or remove earlier saved assessments. Similarly, deleting a history row changes the application's database state; it does not promise secure erasure of all filesystem backups. The implementation should be described through the operations it actually performs.

Robustness also depends on preserving the meaning of failure responses. Invalid input should not be presented as a successful low-risk assessment. A valid heuristic fallback should clearly report the unavailable model. A successful assessment with a storage warning should remain readable. These distinctions allow a user to decide whether to correct the input, restore the model, or retry saving.

5.8 DISCUSSION OF THE MEASURED OUTCOME

The trained model correctly classifies 7,091 of 7,882 held-out records. Its higher recall than precision means that, on this particular test partition, it captures a larger fraction of phishing records at the cost of flagging some benign records. The 319 false negatives and 472 false positives describe different costs and should remain visible when considering how the tool is used.

The application result is broader than a model label because it includes rules, structure checks, and an explanation. However, this integration has not been evaluated in a separate comparative experiment. The measured result supports a functioning local classifier and a reproducible implementation; it does not establish a measured advantage of the hybrid score over the model alone.

The practical contribution is therefore the combination of a working assessment interface with traceable data and numerical evidence. The report preserves the exact feature order, dataset identity, split counts, model parameters, and test transcript so that the reported result can be checked. Stronger claims about future URLs require the additional experiments described in Chapter 6.

What the combined evidence supports

The project provides three kinds of evidence: automated checks of software behaviour, a reproduced historical classifier evaluation, and worked application outputs. Together, they show that the components are connected and that the supplied model produces the documented results. Each evidence type has a different unit of observation, so their counts should remain separate. Twenty-one passing tests are not twenty-one additional classification samples.

The three demonstration outputs establish that the final scoring bands are reachable and that their arithmetic can be traced. They do not measure how often the hybrid system assigns the correct band. A dedicated hybrid evaluation would need an explicit definition of the expected outcome for each band, a comparison policy for Suspicious results, and a labelled assessment set selected for that purpose.

The principal engineering outcome is a report that exposes the decision path: validated input, extracted properties, heuristic reasons, model status, combined score, and persistence status. The principal experimental outcome is the measured binary performance on the held-out subset. Keeping both outcomes visible makes the report useful for implementation review while preserving the limits of its experimental conclusions.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY

Secure Web Link Analyzer provides a working local application for assessing warning patterns in URL text before a destination is opened. The implementation connects validation, a shared 24 feature representation, explainable rules, structural heuristics, a trained Random Forest, a browser report, and optional SQLite history. It also exposes the status needed to understand whether a result used the model and whether it was saved successfully.

The project's measured ML result is reproducible from the included artifacts. A balanced 40,000 row DEPHIDES sample was split by the implemented registered domain grouping rule into 32,118 training records and 7,882 test records. The included classifier achieved 89.96% accuracy, 88.29% precision, 91.77% recall, an F1 score of 0.9000, and ROC AUC of 0.9677 on that held out subset. The confusion matrix contains 472 false positives and 319 false negatives, making the remaining errors visible rather than hiding them behind a single score.

The current verification also confirms that all 21 supplied automated tests pass. These checks cover the analysis and API paths, the included model, data provenance, and metric reproduction. The source listings and the complete code package make the main implementation available for review and further development.

How the parts fit together

The completed workflow connects a user-facing question to inspectable intermediate evidence. Validation defines what the application accepts. Feature extraction gives the rules and model a shared representation. Scoring converts their outputs into a consistent display, while the interface and optional history make the assessment available for review. This connection is the main system contribution beyond training a classifier in isolation.

The report also provides enough operational detail to distinguish a working demonstration from an unexplained screenshot. The input examples, score calculations, setup commands, metadata, and verification record describe how the documented outputs were obtained. They give a future maintainer a starting point for checking a changed environment or evaluating a revised model.

6.2 OBJECTIVES REVISITED

Table 6.1: Project objectives and observed outcomes.

Objective	Observed outcome	Status
Validate and normalize URLs	Explicit policy and invalid-input tests	Implemented
Use a consistent feature representation	24 shared ordered features and schema checks	Implemented
Combine rules and a real ML model	Transparent weighted calculation and fallback	Implemented
Support single, batch, and private checks	API routes and history-control tests	Implemented
Make results reproducible	Pinned dataset source, hashes, metrics, and tests	Implemented
Establish wider detection effectiveness	Independent data and hybrid ablation still needed	Future evaluation

Assessing completion against the objectives

The pre-visit objective is addressed by an analysis path that treats the submitted destination as text. The interpretability objective is addressed through named checks, component scores, and explicit model status. The ML objective is supported by the included artifact and reproducible held-out evaluation. The usability objective is represented by single and batch checks, readable reports, and optional history.

These objectives have different evidence requirements. Source review and targeted tests support the absence of destination requests in the exercised paths. Reproduced metrics support the historical classifier claim. A formal study would still be needed to measure whether users understand warnings better or make safer decisions. The current completion assessment therefore concerns the implemented project objectives and their recorded verification.

6.3 LIMITATIONS

The project reduces exposure by treating destinations as text, validating payloads, bounding requests, redacting recognized secrets, using parameterized SQL, and displaying clear model and persistence status. Browser policy headers and output escaping support the local interface. These controls address specific implementation risks rather than establishing a general security guarantee.

Table 6.2: Limitations affecting interpretation and deployment.

Limitation	Effect on interpretation or operation
Historical single source data	Results may not transfer to current browsing
Balanced sample	Measured precision may differ at real world prevalence
Approximate registered domains	Some related URLs may be grouped imperfectly
Uncalibrated model probability	A score of 90 is not a demonstrated 90% chance of harm
Static heuristic lists	Coverage and false alarms depend on maintained choices
URL text only	Page content, redirects, and later compromise remain unknown
No authentication	Local history is not separated by user accounts
In memory rate limiter	Limits are process local and reset on restart

The tests support the claimed implementation behaviour, and the held out reproduction supports the stated classifier metrics. They do not establish optimal weights, production readiness, or superiority over other methods. The strongest justified conclusion is that this supplied local prototype integrates a real model with explainable URL heuristics and retains enough evidence to reproduce its measured classifier result.

Consequences for practical use

Several limitations interact. Historical labels and a single data source affect the relevance of the learned patterns. Approximate domain grouping affects how strongly the split separates related addresses. Uncalibrated probabilities affect the interpretation of the numerical model output. The final weighting then adds a policy whose overall classification quality has not received a separate benchmark.

For these reasons, the displayed assessment is most useful as a prompt to inspect the address and its warning signs. It does not establish the current state of a remote page. A simple-looking address can lead to harmful content, and a complex address can belong to a legitimate service. The system's purpose remains the assessment of observable URL patterns within its stated evidence boundary.

6.4 FUTURE WORK

Table 6.3: Proposed experiments and the questions they address.

Proposed change	Question it should answer
Independent and later data	Does performance persist outside the original source and period?
Full suffix handling	Does more accurate domain grouping change leakage and results?
Rule, ML, and hybrid ablation	Does combining components improve the chosen objective?
Feature removal experiment	How much does HTTPS or another dominant feature affect transfer?
Probability calibration study	Can model output support more reliable probability interpretation?
Repeated grouped evaluation	How sensitive are metrics to the selected split?
Error category review	Which patterns produce false negatives and false positives?

For a shared deployment, add authentication, authorization, request origin protections, a production server configuration, and an explicit retention policy. Replace the process local limiter if requests are distributed across workers. Review model distribution and integrity controls before allowing model replacement through any user facing feature.

A browser extension or message inspection integration could make pre visit checks easier to access, but it would introduce new permissions and interface constraints. External reputation or destination content analysis would also change the current privacy and trust boundary. Such extensions should be designed as explicit new modes with their own evidence, rather than silently added to the local URL text workflow.

Order of the next experiments

The first experimental priority is to define the intended use and the error tradeoff. That definition should determine the validation procedure before weights or thresholds are adjusted. The next step is to add independent or later labelled data with documented provenance, then measure performance under a grouping procedure appropriate to the new collection.

Component comparisons can follow on the same partitions: rules alone, the trained classifier alone, and the hybrid policy. The evaluation should specify how a Suspicious outcome is counted and whether users are expected to review it. Without that definition, a three-band output cannot be reduced to a meaningful accuracy value simply by reusing the binary model's metric.

Application improvements should follow the intended operating context. A personal local demonstration has different storage and access requirements from a shared service. If shared use is introduced, authentication, ownership of history, retention,

deployment configuration, and rate limiting need an explicit design. These changes should be reviewed together because they alter who can submit, inspect, and retain assessment data.

6.5 CONCLUSION

The report distinguishes completed work from future evaluation. The three band hybrid score is implemented and its arithmetic is tested, but its end to end classification quality has not been measured in a separate benchmark. The model probability is uncalibrated. Historical labels, one collection source, approximate domain grouping, and URL only evidence limit the conclusions that can be drawn about live destinations.

Within those boundaries, the project meets its central objective: it gives the user an understandable, locally computed assessment with traceable inputs and a real machine learning component. The most valuable next step is a stronger evaluation protocol using independent data, component ablations, and a reserved validation set. Application expansion should preserve the same clarity about what the system observes, what it stores, and what its results can support.

Closing observation

The project leaves a clear path for further work: preserve the reproducible baseline, evaluate proposed changes under a stated protocol, and update the application only when the resulting behaviour is understood. The existing system demonstrates a complete URL assessment workflow with a real trained model and visible reasoning. Its value rests on that traceable implementation and measured evidence, together with an accurate account of what remains to be evaluated.

REFERENCES

[1] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL Detection using Machine Learning: A Survey,” arXiv:1701.07179, 2017. Primary research background for URL representations and detection methods. https://arxiv.org/abs/1701.07179

[2] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” arXiv:1802.03162, 2018. Primary source for learned character and word URL representations. https://arxiv.org/abs/1802.03162

[3] E. Buber, O. K. Sahingoz, and E. Kugu, DEPHIDES project repository and dataset, MIT licence. Source revision used by this project: e66ef86fc93c6881c194833d479db9862354d7c4. https://github.com/ebubekirbbr/dephides

[4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, no. 85, pp. 2825-2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[5] scikit-learn developers, RandomForestClassifier API reference, version 1.8.0. Used to identify the estimator behaviour and parameter meanings. https://scikit-learn.org/1.8/modules/gener ated/sklearn.ensemble.RandomForestClassifier.html

[6] scikit-learn developers, GroupShuffleSplit API reference, version 1.8.0. The split size concerns groups rather than an exact number of samples. https://scikit-learn.org/1.8/modules/generat ed/sklearn.model_selection.GroupShuffleSplit.html

[7] Python Software Foundation, urllib.parse documentation, Python 3.12. Used for URL component parsing and its validation limitations. https://docs.python.org/3.12/library/urllib.parse.html

[8] Python Software Foundation, sqlite3 documentation, Python 3.12. Reference for local database connections and parameterized operations. https://docs.python.org/3.12/library/sqlite3.html

[9] scikit-learn developers, Model persistence, version 1.8.0. Reference for trusted pickle based artifacts and environment compatibility. https://scikit-learn.org/1.8/model_persistence.html

[10] scikit-learn developers, Permutation feature importance, version 1.8.0. Reference for interpreting model importance and complementary held out analysis. https://scikit-learn.org/1.8/modules/permutation_importance.html

[11] Pallets Projects, Deploying to Production, Flask 3.1 documentation. Reference for separating development server use from a production deployment. https://flask.palletsprojects.com/en/stable/deploying/

[P1] Secure Web Link Analyzer project source supplied with this report. Principal implementation files: backend/analyzer.py, app.py, feature_extractor.py, ml_model.py, train_model.py, and prepare_dataset.py; frontend/index.html and frontend/static assets; database/database.sql. Health endpoint version 2.1.1.

[P2] Supplied dataset.csv, dataset_metadata.json, model_metrics.json, and trained_model.pkl. Model training timestamp: 2026-09-26T16:47:55.221136+00:00. These files establish the dataset provenance, exact feature order, split counts, estimator settings, and measured classifier results.

[P3] Supplied tests/test_app.py and tests/test_trained_model.py, executed on 14 October 2026. Result: 21 tests passed. The original transcript and verification examples are included under report_evidence in the accompanying code package.

[P4] ICONCISE_Corrected.docx, supplied project paper. Used for project context and consistency of title and authors. Implementation and numeric claims in this report are grounded in the current source and machine readable evaluation files.

Online references were checked on 14 October 2026; selected research and model documentation was reviewed again on 22 October 2026. Project artifacts provide the evidence for implementation and measured results. External works provide technical context; their experiments are not presented as experiments performed by this project.

APPENDIX A - REPRODUCIBILITY AND REPOSITORY LAYOUT

REPRODUCING THE RESULTS

Extract the accompanying source package into a writable folder. Python 3.12 is the version used for verification. The project documentation supports Python 3.11 or newer, but the included model should be used with the pinned ML package versions. The Windows launcher creates a local environment, installs dependencies, checks model readiness, and starts the application.

For a manual Windows setup, run these commands from the extracted project folder. Keep the terminal open while using the browser interface.

```
py -3.12 -m venv .venv
.venv\Scripts\activate
python -m pip install -r backend/requirements.txt
python backend/check_ml_setup.py
python backend/app.py
```

On Linux or macOS, the corresponding environment activation is shown below. The application commands are otherwise the same.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r backend/requirements.txt
python backend/check_ml_setup.py
python backend/app.py
```

Open http://127.0.0.1:5000 in a browser. Do not open frontend/index.html directly from the filesystem, because the interface expects its local API and static assets to be served by Flask.

Table A.1: Runtime configuration and artifact locations.

Setting	Default	Purpose
LINKGUARD_PORT	5000	Change the local listening port
LINKGUARD_DATABASE_PATH	database/secure_link_analyzer.db	Use a separate writable history file
Model file	backend/trained_model.pkl	Supplied classifier artifact
Dataset file	backend/dataset.csv	Reproduction and optional retraining data

The health endpoint should show the service as online and the model as available. The supplied model is ready for inference; retraining is optional. The Flask development

server is appropriate for development and demonstration, while a real deployment needs an appropriate server and deployment controls [11].

REPOSITORY LAYOUT

Table A.2: Files supplied with the runnable code package.

Location	Contents
backend/	Analysis, feature extraction, model inference, API, data preparation, and training
frontend/	Browser interface, styles, scripts, and icons
database/	SQL schema; history is created locally when needed
tests/	Analysis, API, and included-model verification
backend/dataset.csv	Prepared 40,000-row labelled dataset
backend/trained_model.pkl	Supplied trusted model artifact
backend/model_metrics.json	Recorded training settings, split counts, and performance
report_evidence/	Test transcript, example assessments, and source hashes

The full code package supplied with the earlier report remains the runnable project. It contains the source, dataset, model, dependency files, licence notices, and tests. The historical user database and generated cache files were excluded. Chapter 4 contains short excerpts for examination; the source package contains the complete files.

MAINTENANCE

Keep the feature schema, analyzer, model artifact, dataset metadata, and metrics as a compatible set. When feature definitions change, retrain the model and rerun the integration tests. When only interface text changes, verify that the displayed meaning still matches the API fields and thresholds. The legacy database ml_score field deserves particular care because it contains the structure heuristic value.

Review static term, TLD, shortener, brand, and extension lists as documented project configuration. Adding entries should be accompanied by representative tests and an evaluation of possible false positives. A longer list is not automatically a better detector. Preserve dataset attribution and licence notices when repackaging or replacing training data.

AUDITING A NUMERIC CLAIM

Begin with model_metrics.json for the stored value, then inspect train_model.py for its calculation and tests/test_trained_model.py for reproduction. Use dataset_metadata.json for provenance and cleaning counts. Compare the dataset hash and model timestamp before treating a local run as the same experiment. The recorded transcript identifies the dependency versions used for report verification.

For a numerical cross-check, start with the confusion matrix. The four counts are 3,532 true negatives, 472 false positives, 319 false negatives, and 3,559 true positives. Their sum is 7,882. Accuracy is (3,532 + 3,559) / 7,882, precision is 3,559 / (3,559 + 472), and recall is 3,559 / (3,559 + 319). These calculations reproduce the printed results after rounding.

For an application cross-check, run one of the demonstration addresses with history saving disabled and inspect the returned score_breakdown. Compare the direct rule sum, structural contribution, heuristic score, model score, and weights before comparing the final verdict. This distinguishes an arithmetic or integration issue from a change in the underlying model prediction.

Keep the original model, dataset, metrics, and dependency versions together when preparing a demonstration. A newly trained model is a new experiment even if its filename is unchanged. Record its new timestamp and evaluation output, and update any affected report tables before presenting the new result.